

Mixing-period tag recaptures in MULTIFAN-CL: a simulation study

Comparing `tag_flags(i,2)` settings against likelihood-based alternatives

2026-08-05

Table of contents

1	Executive summary	2
2	Scope and methods	4
2.1	Purpose	4
2.2	Setup	5
2.3	Operating model	7
2.4	The mixing-period step	8
2.5	The estimators	9
3	Experiment A: reporting-rate misspecification	12
3.1	Mechanism: what happens to the tagged population	12
3.2	Simulation	14
3.2.1	Bias in estimated fishing mortality	15
3.2.2	Do the flag settings bracket the truth?	17
4	Experiment B: mixing length and mixing-period availability	19
4.1	Bias across the design	21
4.2	The <code>flag0</code> robustness result	25
4.3	Does bracketing ever occur?	26
4.4	The survival-floor condition	29
4.5	Recovery of the availability multiplier	31
4.6	Consistency with Experiment A	32
5	Experiment C: a realistic reporting process and the survival-floor failure mode	33
5.1	Machinery	34
5.2	Design	38

5.3	How often does the floor engage?	41
5.4	What it costs	44
5.5	Does the mixing-channel decomposition survive?	49
6	Experiment D: the mixing-period recapture fraction	50
6.1	Why ψ should govern the <code>flag1</code> bias	51
6.2	Simulation	52
6.3	Is ψ sufficient?	55
6.4	Reading this for a real assessment	58
7	Experiment E: release-group size and the noisy-inversion bias	59
8	What the results show	66
9	What to check in an assessment that fits to tags	69
10	Guidance on <code>tag_flags(i,2)</code>	70
11	Limitations	72
12	Suggested extensions	74
13	Session information	75

1 Executive summary

MULTIFAN-CL handles tag recaptures during the mixing period by inverting the observed returns rather than predicting them. `tag_flags(i,2)` selects the target: the default raises reported returns by the assumed reporting rate and removes that many tags; the alternative removes the reported returns at face value, which amounts to assuming complete reporting. Neither scores the mixing-period data in the likelihood, so under both settings those data fix the tagged population state with no uncertainty attached and no ability for the model to object. **Both treatments are wrong, and the choice cannot be avoided in any model that fits to tags.** This study simulates both, together with two likelihood-based alternatives, across reporting-rate misspecification, mixing-period length, tagged-fish availability during mixing, a realistic clumped reporting process, and release-group size.

The default is unbiased under three conditions and not otherwise: the assumed reporting rate is correct, the survival floor is not engaging, and release groups are large. Given those, it recovers F across every combination of mixing length and tagged-fish availability tested, which is a genuine strength — the deterministic inversion never needs to know how available tagged fish were during mixing. **The alternative is biased even when everything else is specified perfectly, but its bias is at least largely predictable.**

It is governed by ψ , the proportion of each release group recaptured within `tag_flags(i,1)` periods of release: a few percent at $\psi = 0.05$, roughly a third of F at $\psi = 0.3$, half or more by $\psi = 0.5$. That fraction is computable from the `.tag` file in advance, provided the reported returns are raised by the assumed reporting rate first.

Where the default’s conditions fail, the errors are consequential and hard to see. When the realised catch fraction in a mixing period exceeds roughly four-fifths of the assumed reporting rate, `posfun` silently caps the removal, the default then *under-removes*, and F is biased downward and biomass upward — the same direction as the alternative it is supposed to improve on. At small release-group sizes the inversion’s convexity puts weight in the upper tail of F instead: a minority of groups return estimates many times the truth while the median at the same cell says nothing at all, so this second failure is invisible to the diagnostics an assessment normally reports. Nor do the two settings bound the problem between them. Because the assumed reporting rate is below one, the default always removes more tags and always returns the larger F , so the pair is structurally one-sided: it cannot contain the truth when the assumed rate is too high, and where it does contain it the interval is often wider than the quantity being estimated. Running both as grid levels spans an assumption without bounding anything. Note too that the assumed reporting rate is itself the first-order channel in most configurations examined — larger than the flag choice — so a biomass shift should not be attributed to the flag without knowing what the reporting rates did.

This matters now because reporting-rate estimates are moving downward. The tag-seeding analysis prepared for the 2026 assessments (SC22-SA-IP-05) puts the purse seine reporting-rate priors for bigeye at means of 0.496 to 0.528 across regions 2 to 4, between 15 and 24 per cent below what the 2023 assessments would have used under the same region and fishery definitions. The survival-floor condition is reached routinely at an assumed rate of 0.2 and is reachable at 0.5 when tagged fish are highly available during mixing, so the bigeye assessment sits at the boundary of the default’s second condition rather than comfortably inside it, and it arrived there by moving in the direction that makes the condition harder to satisfy.

There is a treatment that removes the dilemma, and it is not available to the 2026 assessments. Predicting mixing-period returns and scoring them in the likelihood, with the reporting rate free to move against its tag-seeding prior, puts those data on the same footing as post-mixing returns and inverts nothing. This study implements that treatment (`forward_X`) alongside the two flag settings, and it recovers F under the reporting-rate misspecification that biases both of them. MULTIFAN-CL cannot currently do this: the mixing-period returns do not enter the tag likelihood under either flag setting, so it is a redevelopment request to the MULTIFAN-CL team rather than a configuration choice. It is also not free. With many small release groups it was the least precise of the four estimators tested here — estimating the reporting rate, the availability multiplier and F together needs information a fragmented tagging programme may not carry. That finding comes with a caveat: the prior imposed on the reporting rate in these runs is far looser than the tag-seeding priors an assessment would use, so an unknown share of the imprecision belongs to the prior rather than to the estimator, and the comparison should be re-run before the finding is leaned on.

In the meantime the most useful response is not to pick a setting but to bound how much the tag data are driving the answer at all. We recommend that a configuration omitting tagging data — with the spatial information instead carried by a spatially implicit fleets-as-areas structure — be included in the structural uncertainty grid, and treated as a more informative addition to it than either flag setting. If it produces similar stock status, the mixing-period treatment is a second-order concern and can be documented as such; if it does not, the difference is attributable to a model component that is defective under either setting. It also answers a question managers ask directly: how much of this result depends on the tagging programme? Two of the default’s three conditions — whether the survival floor is liable to engage, and whether release groups are large enough for the large-sample argument to apply — can be checked in advance against the `.tag` file and the flag settings, and [What to check in an assessment that fits to tags](#) sets out how. The third, the correctness of the assumed reporting rate, is the quantity under dispute and cannot be settled from inside the model.

A note on scope. The tag-free, spatially implicit fleets-as-areas recommendation is a reasoned position, not a simulated result — nothing here tests a spatial model or a fleets-as-areas configuration. What the simulation supports is the narrower claim that both `tag_flags(i,2)` settings are biased, that their biases are not bounding, and that the conditions under which each fails are identifiable in advance.

2 Scope and methods

2.1 Purpose

MULTIFAN-CL handles tag recaptures differently before and after the mixing period.

After mixing, the tagged population is projected forward with the population fishing mortality, predicted returns are formed as $E[R] = X \times C$ (reporting rate times true recaptures), and observed returns are scored in the likelihood. Nothing is inverted.

During mixing, `do_newton_raphson_for_tags()` instead solves deterministically for a tag-specific F such that the removal equals a target derived from the observed returns. The flag `tag_flags(i,2)` chooses the target:

Setting	Target removal	Interpretation
<code>tag_flags(i,2) = 0</code> (default)	R_{obs}/X	reported returns raised for non-reporting
<code>tag_flags(i,2) = 1</code>	R_{obs}	reported returns taken at face value

Mixing-period returns contribute **nothing** to the likelihood under either setting. They have full authority over the tagged population state and zero influence on parameter estimation, with no uncertainty attached.

This vignette runs five experiments.

Experiment A holds the *assumed* reporting rate fixed at 0.5 — the value the analyst supplies, whether from tag seeding or by assumption — and varies the *true* reporting rate above, at, and below it. That is the realistic framing: you choose what you assume, and the error runs in a direction you cannot see.

Experiment B adds two further dimensions that the assessment configuration controls directly: the length of the mixing period, `tag_flags(i,1)`, at 2, 3 and 4 quarters; and the relative availability of tagged fish during mixing, including the case where tagged fish are *more* vulnerable than the general population rather than less. The latter is arguably the more realistic case for purse-seine fisheries, where tags are released from fishing vessels into concentrated effort.

Experiment C replaces the clean observation model with a realistic one: small release groups, stochastic recaptures, and reporting clumped at the trip level. This is what makes the survival-floor failure mode reachable at ordinary fishing mortalities, and it identifies the assumed reporting rate as the parameter that controls it.

Experiment D reparameterises the `flag1` result by the fraction of a release group recaptured during mixing, which turns the bias into something an analyst can look up from their own `.tag` file.

Experiment E isolates release-group size, to test whether the residual scatter `flag0` shows at high recapture fractions in Experiment D is Monte Carlo noise or a real small-sample property of inverting a noisy target.

The operating model is deliberately stripped down: one fishery, one region, no age structure, no movement, no tag shedding beyond natural mortality. Experiments A and B use a single large release cohort; C and D use several smaller groups sharing one fishing mortality; E uses a single group of varying size, for the reason given in that section. It is not a fisheries model. It isolates the mixing-period mechanism and nothing else.

2.2 Setup

```
library(data.table)
library(magrittr)
library(ggplot2)

theme_set(theme_bw(base_size = 11))
set.seed(20260802)
```

```

pal <- c(flag0      = "#D85A30",
        flag1      = "#1D9E75",
        forward     = "#7F77DD",
        forward_X   = "#378ADD",
        truth       = "grey30")

```

```

# Threshold defining a "gross overshoot" of the deterministic inversion, as a
# multiple of the true F. Used as the primary tail statistic in Experiment E.
# It has to sit far enough above the truth to be unambiguous, far enough below
# the search ceiling to be insensitive to it (the ceilings are at roughly 33x
# and 333x the truth), and low enough to capture sufficient mass for the
# difference across group sizes to be resolvable at the replicate count used.
E_TAIL_MULT <- 5

```

Formatting helpers. Every number quoted in the prose below is produced by one of these from a fitted object, so the text cannot drift out of step with the results when the design or the replicate count changes.

```

# Percentage, signed by default (biases read better with an explicit sign).
fpc <- function(x, digits = 0, signed = TRUE) {
  sprintf(paste0("%", if (signed) "+" else "", ".", digits, "f%%"), 100 * x)
}

```

```

# Range of a vector, as a percentage: "-41% to +16%". If both ends format
# identically -- which happens whenever a quantity is constant across the cells
# being summarised -- collapse to the single value, so the prose reads "0%"
# rather than the nonsensical "0% to 0%".
fpc_range <- function(x, digits = 0) {
  lo <- fpc(min(x), digits)
  hi <- fpc(max(x), digits)
  if (identical(lo, hi)) lo else paste0(lo, " to ", hi)
}

```

```

fnum <- function(x, digits = 2) formatC(x, format = "f", digits = digits)

```

```

# Range of values already expressed in percentage points: "17-67%". Collapses to
# a single value when both ends round alike, so a quantity that is constant
# across the cells being summarised does not read as the nonsensical "0-0%".
fpp_range <- function(x, digits = 0) {
  lo <- fnum(min(x), digits)
  hi <- fnum(max(x), digits)
  if (identical(lo, hi)) paste0(lo, "%") else paste0(lo, "\u2013", hi, "%")
}

```

```
# Monte Carlo standard error of a sample median (normal approximation).
mcse_median <- function(x) 1.2533 * stats::sd(x) / sqrt(length(x))
```

2.3 Operating model

Discrete quarterly steps. Within a period, tagged fish die from fishing and natural causes following Baranov:

$$C_t = \frac{F_t^T}{Z_t} (1 - e^{-Z_t}) N_t, \quad N_{t+1} = N_t e^{-Z_t}, \quad Z_t = F_t^T + M$$

During the mixing period, tagged fish have a different availability to the fishery than the general population, so $F_t^T = \varphi F_t$; afterwards $\varphi = 1$. Values $\varphi < 1$ mean tagged fish are harder to catch than average (the assumption implicit in the usual justification for a mixing period); $\varphi > 1$ means they are easier to catch, which is what happens when effort is concentrated around release sites. Reported returns are a thinned observation of the true recaptures, $R_t \sim \text{Poisson}(X_{true} C_t)$.

```
baranov <- function(f, n, m) {
  z <- f + m
  (f / z) * (1 - exp(-z)) * n
}

om <- list(
  NO      = 10000, # tags released (known exactly)
  Tmax    = 12,   # quarters at liberty
  M        = 0.20, # natural mortality per quarter
  F_true   = 0.15, # post-mixing fishing mortality per quarter -- the estimand
  X_assumed = 0.50, # the reporting rate the analyst supplies -- fixed in every
                    # experiment except C, which varies it deliberately. The
                    # value is the 2026 bigeye purse seine prior mean (see below).
  nmix     = 2,   # baseline mixing periods, i.e. tag_flags(i,1)
  phi      = 0.50 # baseline availability multiplier during mixing
)

simulate_returns <- function(p, X_true, phi = p$phi, nmix = p$nmix) {
  n <- p$NO
  r <- numeric(p$Tmax)
  for (t in seq_len(p$Tmax)) {
    f_t <- p$F_true * if (t <= nmix) phi else 1
    r[t] <- rpois(1, X_true * baranov(f_t, n, p$M))
  }
}
```

```

      n      <- n * exp(-(f_t + p$M))
    }
    r
  }

```

The assumed reporting rate of 0.50 is not a round number chosen for convenience. The tag-seeding analysis prepared for the 2026 assessments (SC22-SA-IP-05) gives purse seine reporting-rate priors for bigeye with means of 0.496 in region 2, 0.512 in region 3 and 0.528 in region 4, so 0.50 is the operative value for the assessment this work is aimed at and the panels at $X_{assumed} = 0.5$ elsewhere in this document are the ones to read first. Three qualifications travel with that number and are repeated wherever it is used: those are purse seine fisheries only, so the mapping onto a single-fishery operating model is by analogy; they are penalty distributions rather than fixed values, so a real model estimates the rate against them rather than dividing by a constant; and the region 2 penalty was deliberately halved to reflect additional uncertainty, with the prior not recommended for the Indonesian and Philippine domestic fisheries at all.

2.4 The mixing-period step

One function performs the deterministic inversion, and every estimator, every trajectory plot and every diagnostic in this document calls it. This is load-bearing rather than tidiness. If the estimator capped the removal one way — say a hard `min(target, 0.99 * n)` — and the floor diagnostic used an ADMB-style survival floor, the two would advance the tagged population differently as soon as the cap engaged, and the frequency of floor engagement reported would not be the frequency the estimator actually experienced. A single implementation makes that class of discrepancy impossible by construction.

The floor reproduces the MULTIFAN-CL behaviour qualitatively: survival is not allowed below `cut + fringe`, the removal is rescaled to whatever the floored survival implies, and a quadratic penalty is accumulated. The defaults match `age_flags(118)` and `(119)`.

```

CUT      <- 0.20    # age_flags(118) analogue
FRINGE   <- 0.02    # age_flags(119) analogue -- survival floor is CUT + FRINGE
PENWT    <- 1e3     # penalty weight; see note below and the Limitations

# ADMB posfun: smooth floor with a quadratic penalty.
posfun <- function(x, eps) {
  if (x >= eps) return(c(y = x, pen = 0))
  c(y = eps / (2 - x / eps), pen = (x - eps)^2)
}

# Newton solver for the tag F. Mirrors do_newton_raphson_for_tags() and is
# faster than uniroot().
solve_F_newton <- function(target, n, m, iter = 25) {

```

```

if (target <= 0) return(0)
u <- min(target / n, 0.999)
f <- -log(1 - u)                                # exponential-approximation start
for (i in seq_len(iter)) {
  z <- f + m
  e <- exp(-z)
  g <- (f / z) * (1 - e) * n - target
  dg <- n * ((m * (1 - e)) / z^2 + (f / z) * e)
  f <- max(f - g / dg, 1e-10)
}
f
}

# THE single mixing-period step. Returns the realised target, the F that
# achieves it, the penalty incurred, and whether the survival floor engaged.
mix_step <- function(r_t, n, p, mode, cut = CUT, fringe = FRINGE) {
  target <- if (mode == "flag0") r_t / p$X_assumed else r_t
  surv <- 1 - target / n
  floored <- surv < cut + fringe
  pen <- 0
  if (floored) {
    pf <- posfun(surv - fringe, cut)
    surv <- fringe + pf[["y"]]
    pen <- pf[["pen"]]
    target <- n * (1 - surv)
  }
  list(target = target, f = solve_F_newton(target, n, p$M),
        pen = pen, floored = floored)
}

```

Note that for `flag0` and `flag1` the target depends only on the data, never on a parameter, so the penalty is a constant with respect to the parameter vector and cannot move $\hat{F}$. This is not a shortcut: it is a faithful analogue of the `age_flags(117) < 0` case, in which the rescaling happens with no trace in the objective function. It does mean the penalty in this vignette is a bookkeeping device rather than something the optimiser responds to.

2.5 The estimators

All estimate F on the log scale. They differ only in how the mixing periods are handled. Every one uses `X_assumed = 0.5`; none is told the true reporting rate, and none is told φ .

Label	Mixing-period removal	Mixing returns in likelihood?	Free parameters
flag0	$R_t/X_{assumed}$, solved deterministically	no	$\log F$
flag1	R_t , solved deterministically	no	$\log F$
forward	predicted from φF	yes	$\log F, \log \varphi$
forward_X	predicted from φF	yes	$\log F, \log \varphi, \text{logit } X$

forward_X additionally estimates the reporting rate under a lognormal prior centred on the assumed value — the analogue of a tag-seeding prior via `fish_flags(i,35)` and `(i,36)`.

flag0 and flag1 never estimate φ ; they do not need to, because they solve for whatever mixing-period F reproduces their target removal. Whether that makes them robust to φ or merely indifferent to it is one of the questions Experiment B answers.

Because the flag modes have a single free parameter, they are optimised with `optimize()` rather than being handed a three-element simplex with two exactly flat directions. This is both faster and more reliable than handing them to a general-purpose optimiser.

```
nll_tag <- function(par, r, p, mode, nmix = p$nmix, prior_sd = 0.4) {
  f_pop <- exp(par[1])
  phi   <- if (mode %in% c("forward", "forward_X")) exp(par[2]) else 1
  x     <- if (mode == "forward_X") plogis(par[3]) else p$X_assumed

  n <- p$N0
  nll <- 0

  for (t in seq_len(p$Tmax)) {
    premix <- t <= nmix

    if (premix && mode %in% c("flag0", "flag1")) {
      # Deterministic inversion. No likelihood contribution; the penalty is
      # constant in `par` (see note above).
      st <- mix_step(r[t], n, p, mode)
      f_t <- st$f
      nll <- nll + PENWT * st$pen
    } else {
      # Forward prediction, scored against the data.
      f_t <- if (premix) f_pop * phi else f_pop
      mu <- x * baranov(f_t, n, p$M) + 1e-10
      nll <- nll - dpois(r[t], mu, log = TRUE)
    }
  }
}
```

```

    n <- n * exp(-(f_t + p$M))
  }

  if (mode == "forward_X") {
    nll <- nll + 0.5 * ((log(x) - log(p$X_assumed)) / prior_sd)^2
  }
  nll
}

LOGF_INT      <- c(log(1e-3), log(5))      # search interval for the 1-D flag modes
LOGF_INT_WIDE <- c(log(1e-5), log(50))     # stress interval, used in Experiment E
BOUND_TOL     <- 1e-3                     # log-scale tolerance for endpoint contact

# The flag modes are optimised over a bounded interval, so `optimize()` always
# returns a value and a convergence code would be meaningless for them. The
# meaningful diagnostic is whether the returned estimate is sitting on an
# endpoint: if it is, the number reported is a property of the search interval
# rather than of the data, and any statistic sensitive to the tail of the F-hat
# distribution has to be checked against it. Replicates that hit a bound are
# retained in every summary below, not dropped -- discarding them would remove
# exactly the behaviour being characterised -- but they are counted and reported.
at_bound_lf <- function(lf, interval, tol = BOUND_TOL) {
  any(abs(lf - interval) < tol)
}

fit_one <- function(r, p, mode, nmix = p$nmix, interval = LOGF_INT) {
  mode <- as.character(mode)[1L]

  if (mode %in% c("flag0", "flag1")) {
    opt <- stats::optimize(
      function(lf) nll_tag(c(lf, 0, 0), r, p, mode, nmix),
      interval = interval, tol = 1e-10)
    return(data.table(F_hat = exp(opt$minimum), phi_hat = NA_real_,
                      X_hat = p$X_assumed, conv = 0L,
                      at_bound = at_bound_lf(opt$minimum, interval)))
  }

  opt <- stats::optim(
    par = c(log(0.10), 0, qlogis(p$X_assumed)),
    fn = function(par) nll_tag(par, r, p, mode, nmix),
    method = "Nelder-Mead",
    control = list(maxit = 5000, reltol = 1e-12))

```

```

data.table(
  F_hat      = exp(opt$par[1]),
  phi_hat    = exp(opt$par[2]),
  X_hat      = if (mode == "forward_X") plogis(opt$par[3]) else p$X_assumed,
  conv       = opt$convergence,
  at_bound   = FALSE) # unbounded search; `conv` is the diagnostic here
}

```

A note on what the flag modes can and cannot return. The deterministic inversion solves for whatever mixing-period F reproduces its target removal, and that target is a *noisy observation* divided by a fixed constant. When a small release group happens to return an unusually large number of tags in a mixing period, the implied removal can approach the tags available, and the solved F can be arbitrarily large. The search interval is what stops it running away. That makes the upper endpoint a modelling choice with consequences for any mean, and it is why `at_bound` is carried through every experiment and why Experiment E repeats its design at LOGF_INT_WIDE as well.

3 Experiment A: reporting-rate misspecification

The assumed reporting rate is fixed at 0.5. The true rate takes three values, so the assumption is right in one case and wrong in each direction in the others. Mixing length and availability stay at their baseline (`nmix = 2`, $\varphi = 0.5$).

```

scenarios <- data.table(
  X_true = c(0.35, 0.50, 0.65),
  case    = factor(c("true < assumed", "true = assumed", "true > assumed"),
    levels = c("true < assumed", "true = assumed", "true > assumed"))
)

```

3.1 Mechanism: what happens to the tagged population

Before looking at estimates, look at the state. This traces tags at liberty through the mixing period under each treatment, for one realisation per scenario, with F held at its true value. Plotting the ratio to the true trajectory rather than the trajectory itself is what makes the mechanism visible: on an absolute axis, over an order-of-magnitude decline, the three series are indistinguishable.

```

trajectory <- function(r, p, treatment, phi = p$phi, nmix = p$nmix) {
  n      <- p$N0
  out    <- numeric(p$Tmax + 1)
  out[1] <- n

```

```

for (t in seq_len(p$Tmax)) {
  premix <- t <= nmix
  if (premix && treatment %in% c("flag0", "flag1")) {
    f_t <- mix_step(r[t], n, p, treatment)$f
  } else {
    f_t <- p$F_true * if (premix) phi else 1 # "truth"
  }
  n <- n * exp(-(f_t + p$M))
  out[t + 1] <- n
}
data.table(period = 0:p$Tmax, N = out, treatment = treatment)
}

set.seed(11)
traj <- scenarios[, {
  r <- simulate_returns(om, X_true)
  c("truth", "flag0", "flag1") %>%
    lapply(trajjectory, r = r, p = om) %>%
    rbindlist()
}, by = .(case, X_true)]

traj[, treatment := factor(treatment, levels = c("truth", "flag0", "flag1"))]

# Express each treatment relative to the truth within (case, period).
traj[, N_rel := N / N[treatment == "truth"], by = .(case, period)]

ggplot(traj[treatment != "truth"], aes(period, N_rel, colour = treatment)) +
  geom_hline(yintercept = 1, linetype = 2, colour = "grey40") +
  geom_vline(xintercept = om$nmix, linetype = 2, colour = "grey60") +
  geom_line(linewidth = 0.9) +
  facet_wrap(~ case) +
  scale_x_continuous(breaks = seq(0, om$Tmax, by = 3)) +
  scale_colour_manual(values = pal) +
  labs(x = "Quarters at liberty", y = "Tags at liberty / truth", colour = NULL,
       subtitle = paste0("Assumed reporting rate = ", om$X_assumed,
                          " in all panels; only the truth changes"))

```

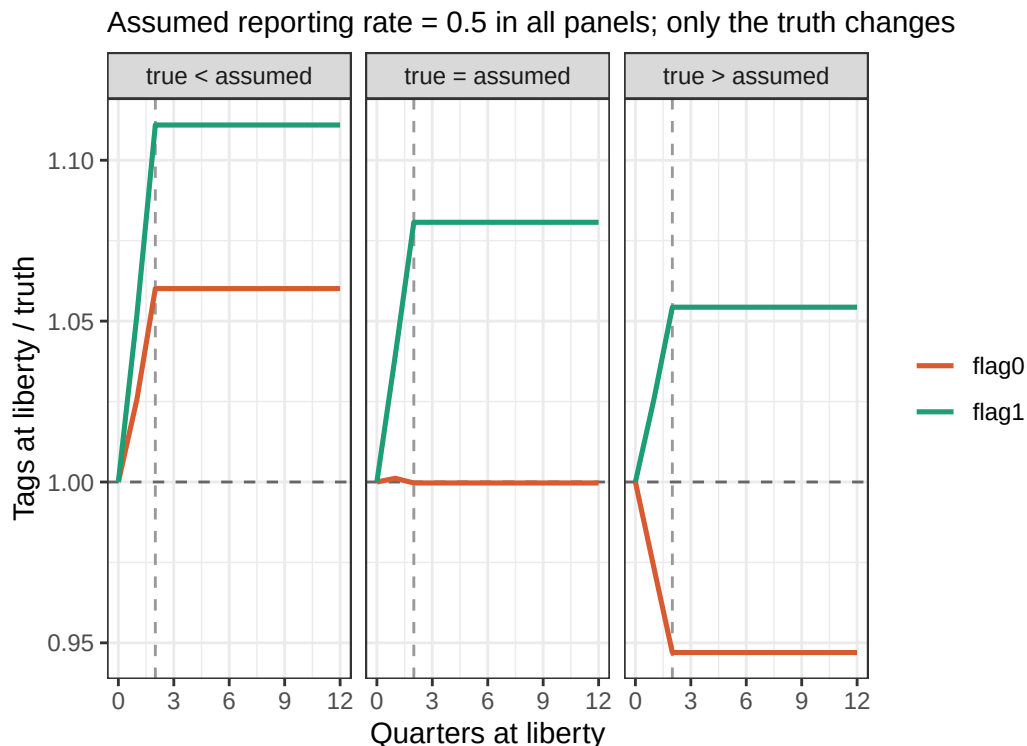


Figure 1: Tags at liberty under each mixing-period treatment, relative to the truth. All divergence is created within the first two quarters (dashed line) and is then carried, unchanged, through the rest of the series. `flag1` always sits above `flag0` because it never raises the returns.

When the true reporting rate is *below* what we assume, observed returns are sparse relative to the true recaptures, and `flag0` divides by too large a number — it under-removes, so the ratio sits above 1. When the truth is *above* what we assume, `flag0` over-removes. `flag1` never raises the returns at all, so it always removes the fewest tags and carries the largest cohort into the informative periods. The flat sections after quarter 2 are the point: the error is created entirely within the mixing period and is then locked in.

3.2 Simulation

```
n_rep <- 200
modes <- c("flag0", "flag1", "forward", "forward_X")

grid_a <- CJ(rep = seq_len(n_rep), case = scenarios$case, mode = modes,
             sorted = FALSE) %>%
```

```

merge(scenarios, by = "case")

res_a <- grid_a[, {
  # One dataset per (rep, case), shared across estimators via a fixed seed,
  # so the four fits are paired and directly comparable.
  set.seed(as.integer(1e6 * X_true) + rep)
  r <- simulate_returns(om, X_true)
  fit_one(r, om, mode = mode)
}, by = .(rep, case, X_true, mode)]

res_a[, `:=`(mode = factor(mode, levels = modes),
  rel_bias = F_hat / om$F_true - 1)]

# Counts flagged fits, not just non-convergent ones: for the bounded 1-D modes
# `conv` is 0 by construction, so a table of `conv != 0` alone would report
# zeros whatever the estimator did.
knitr::kable(
  dcast(res_a[, .(flagged = sum(conv != 0 | at_bound)), by = .(case, mode)],
    case ~ mode, value.var = "flagged"),
  caption = paste0("Experiment A: fits that either failed to converge or ",
    "returned an estimate on the edge of the search interval, ",
    "out of ", n_rep, " per cell."))

```

Table 3: Experiment A: fits that either failed to converge or returned an estimate on the edge of the search interval, out of 200 per cell.

case	flag0	flag1	forward	forward_X
true < assumed	0	0	0	0
true = assumed	0	0	0	0
true > assumed	0	0	0	0

3.2.1 Bias in estimated fishing mortality

```

summ_a <- res_a[conv == 0, .(
  median_F = median(F_hat),
  rel_bias = median(F_hat) / om$F_true - 1,
  mcse = mcse_median(F_hat) / om$F_true
), by = .(case, X_true, mode)][order(X_true, mode)]

knitr::kable(summ_a, digits = 3,
  col.names = c("Case", "True RR", "Estimator",
    "Median F-hat", "Relative bias", "MCSE"))

```

Case	True RR	Estimator	Median F-hat	Relative bias	MCSE
true < assumed	0.35	flag0	0.088	-0.414	0.002
true < assumed	0.35	flag1	0.082	-0.452	0.002
true < assumed	0.35	forward	0.088	-0.413	0.002
true < assumed	0.35	forward_X	0.148	-0.012	0.009
true = assumed	0.50	flag0	0.149	-0.004	0.004
true = assumed	0.50	flag1	0.135	-0.098	0.003
true = assumed	0.50	forward	0.150	-0.003	0.004
true = assumed	0.50	forward_X	0.150	0.002	0.008
true > assumed	0.65	flag0	0.211	0.409	0.004
true > assumed	0.65	flag1	0.191	0.272	0.004
true > assumed	0.65	forward	0.209	0.396	0.004
true > assumed	0.65	forward_X	0.150	0.000	0.008

```

ggplot(res_a[conv == 0], aes(mode, rel_bias, fill = mode)) +
  geom_hline(yintercept = 0, linetype = 2, colour = "grey40") +
  geom_boxplot(outlier.size = 0.6, alpha = 0.85, show.legend = FALSE) +
  facet_wrap(~ case) +
  scale_x_discrete(guide = guide_axis(angle = 30)) +
  scale_y_continuous(labels = scales::percent) +
  scale_fill_manual(values = pal) +
  labs(x = NULL, y = "Relative bias in F-hat")

```

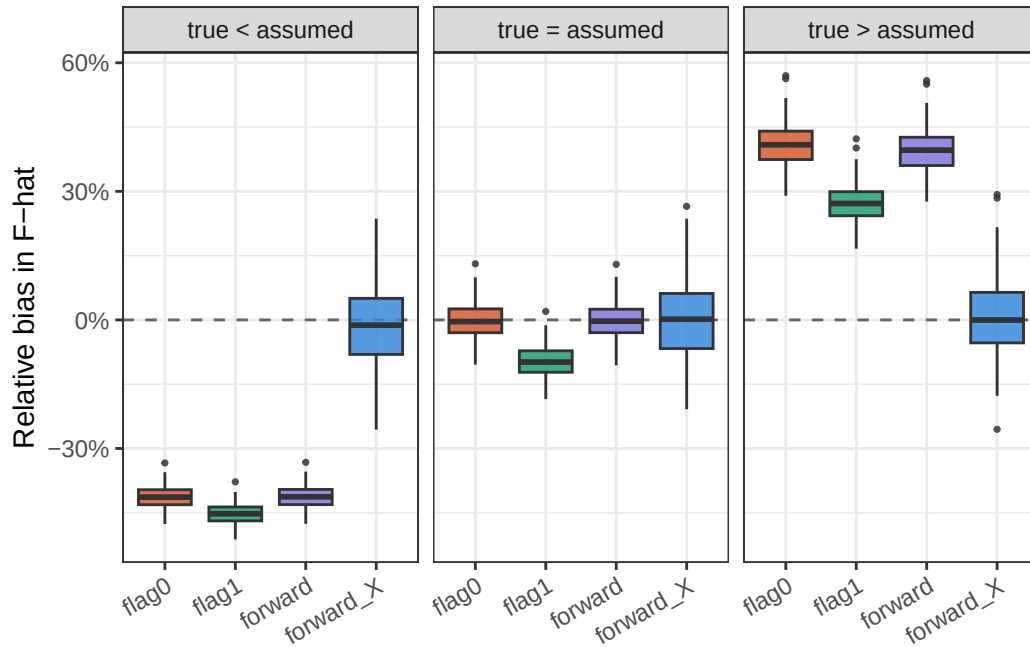


Figure 2: Relative bias in estimated F with the assumed reporting rate fixed at 0.5. `flag0` and `flag1` sit on the same side of zero in every panel.

3.2.2 Do the flag settings bracket the truth?

```
bracket_check <- function(dt, f_true) {
  id <- setdiff(names(dt), c("mode", "F_hat", "conv"))
  keep <- dt[mode %in% c("flag0", "flag1")]

  # Keep only replicates where BOTH flag modes converged. Filtering per mode
  # and then reshaping silently produces NA and poisons the cell mean.
  ok <- keep[, .(both = all(conv == 0) & .N == 2L), by = id][both == TRUE]
  keep <- merge(keep, ok[, ..id], by = id)

  w <- dcast(keep, stats::as.formula(paste(paste(id, collapse = " + "), "~ mode")),
    value.var = "F_hat")
  w[, `:=`(lo = pmin(flag0, flag1), hi = pmax(flag0, flag1))]
  w[, brackets := f_true >= lo & f_true <= hi]
  stopifnot(!anyNA(w$brackets))
  w[]
}
```

```
wide_a <- bracket_check(res_a[, .(rep, case, X_true, mode, F_hat, conv)], om$F_true)
```

```
knitr::kable(
  wide_a[, .(`flag1 (median)` = median(flag1),
             `flag0 (median)` = median(flag0),
             `true F`         = om$F_true,
             `% bracketing`    = 100 * mean(brackets)),
    by = .(case, X_true)][order(X_true)],
  digits = 3)
```

case	X_true	flag1 (median)	flag0 (median)	true F	% bracketing
true < assumed	0.35	0.082	0.088	0.15	0.0
true = assumed	0.50	0.135	0.149	0.15	46.5
true > assumed	0.65	0.191	0.211	0.15	0.0

```
ggplot(wide_a[rep <= 60], aes(x = rep)) +
  geom_linerange(aes(ymin = lo, ymax = hi), colour = "grey55", linewidth = 0.5) +
  geom_point(aes(y = flag0, colour = "flag0"), size = 0.9) +
  geom_point(aes(y = flag1, colour = "flag1"), size = 0.9) +
  geom_hline(yintercept = om$F_true, linetype = 2) +
  facet_wrap(~ case) +
  scale_colour_manual(values = pal) +
  labs(x = "Replicate", y = expression(hat(F)), colour = NULL)
```

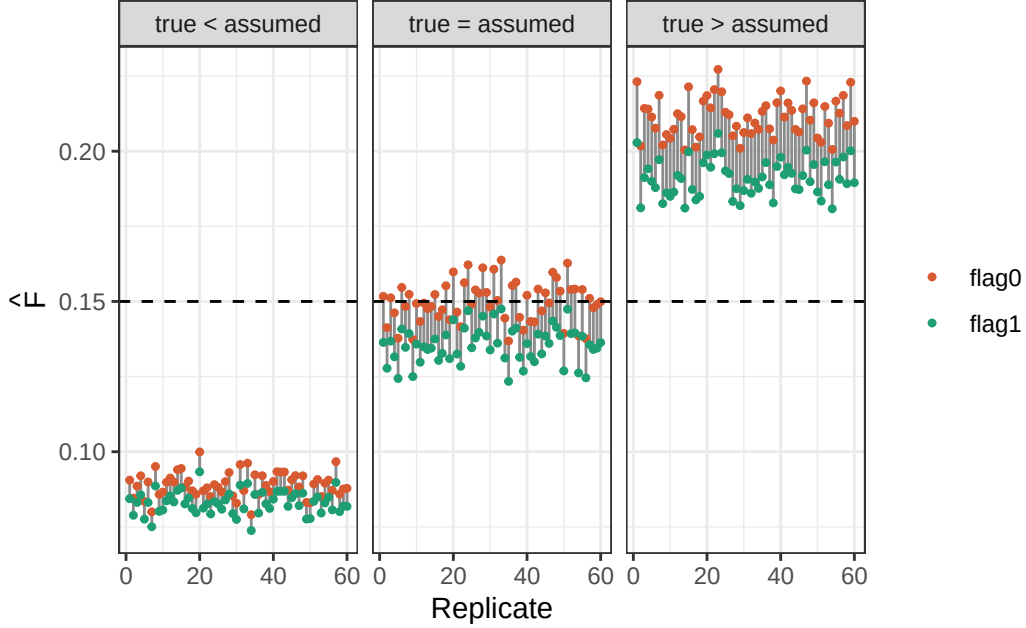


Figure 3: Interval spanned by the two flag settings, first 60 replicates. Dashed line is the true F .

At the baseline configuration the two settings sit on the same side of the truth in both misspecified cases (0.0% bracketing). `flag1` is uniformly the *lower* of the two, so the pair spans an interval — but that interval lies wholly above or wholly below the truth. Note that being the lower endpoint is not the same as being the closer one: in the `true < assumed` case `flag1` is further from the truth than `flag0`, not less extreme.

The one case with non-zero bracketing is the case where the reporting rate is assumed *correctly*, at 46.5%. That is worth reading carefully, because it is not evidence that the pair works. When X is right, `flag0` is unbiased, so it sits on the truth and Monte Carlo noise puts it above the truth roughly half the time. The “bracket” is really just `flag0` landing on the answer, with `flag1` some distance below it. A construct that only contains the truth when one of its two endpoints is already unbiased is not bounding anything.

Experiment B shows why the asymmetry between the two misspecified cases is structural rather than incidental.

4 Experiment B: mixing length and mixing-period availability

Two things the assessment configuration controls directly are added to the design:

- **Mixing length** $\text{nmix} \in \{2, 3, 4\}$ quarters — this is `tag_flags(i,1)`, chosen by the analyst.
- **Availability during mixing** $\varphi \in \{0.5, 1.0, 2.0\}$ — tagged fish half as catchable as average, equally catchable, or twice as catchable. The last case represents concentrated effort around release sites and is *not* exotic for purse-seine tagging.

These are crossed with the same three reporting-rate scenarios, giving 27 cells. Note that φ is a property of the world, not a setting: no estimator is told its value, and only the forward parameterisations even have a parameter for it.

Runtime note: this design is $3 \times 3 \times 3 \times 4 \times \text{n_rep_b}$ model fits. Half of those are one-dimensional and cheap; the forward modes are the cost. Expect a few minutes. Drop `n_rep_b` to 20 for a quick pass; the medians get noisier — watch the MCSE column — but the patterns hold.

```
n_rep_b <- 60          # raise for smoother medians; runtime scales linearly

design_b <- CJ(nmix    = c(2L, 3L, 4L),
              phi      = c(0.5, 1.0, 2.0),
              X_true    = scenarios$X_true,
              mode      = modes,
              rep       = seq_len(n_rep_b), sorted = FALSE) %>%
  merge(scenarios, by = "X_true")

# Seed from a group index rather than an arithmetic combination of the factor
# levels, which is collision-prone as soon as a level or the replicate count
# changes.
design_b[, cell := .GRP, by = .(nmix, phi, X_true)]

res_b <- design_b[, {
  set.seed(1000L * cell + rep)
  r <- simulate_returns(om, X_true, phi = phi, nmix = nmix)
  fit_one(r, om, mode = mode, nmix = nmix)
}, by = .(rep, cell, nmix, phi, X_true, case, mode)]

res_b[, `:=`(mode      = factor(mode, levels = modes),
            rel_bias = F_hat / om$F_true - 1)]

knitr::kable(
  dcast(res_b[, .(flagged = sum(conv != 0 | at_bound)), by = .(case, mode)],
    case ~ mode, value.var = "flagged"),
  caption = paste0("Experiment B: fits that either failed to converge or ",
    "returned an estimate on the edge of the search interval, ",
    "out of ", n_rep_b * 9, " per (case, estimator)."))
```

Table 6: Experiment B: fits that either failed to converge or returned an estimate on the edge of the search interval, out of 540 per (case, estimator).

case	flag0	flag1	forward	forward_X
true < assumed	0	0	0	0
true = assumed	0	0	0	0
true > assumed	0	0	0	0

4.1 Bias across the design

```
summ_b <- res_b[conv == 0, .(rel_bias = median(F_hat) / om$F_true - 1,
                                mcse      = mcse_median(F_hat) / om$F_true,
                                n          = .N),
                        by = .(nmix, phi, case, mode)]

ggplot(summ_b, aes(factor(nmix), rel_bias, colour = mode, group = mode)) +
  geom_hline(yintercept = 0, linetype = 2, colour = "grey40") +
  geom_line(linewidth = 0.7) +
  geom_point(size = 1.8) +
  facet_grid(phi ~ case,
             labeller = labeller(phi = function(x) paste0("phi = ", x))) +
  scale_y_continuous(labels = scales::percent) +
  scale_colour_manual(values = pal) +
  labs(x = "Mixing periods (nmix)", y = "Median relative bias in F-hat",
       colour = NULL)
```

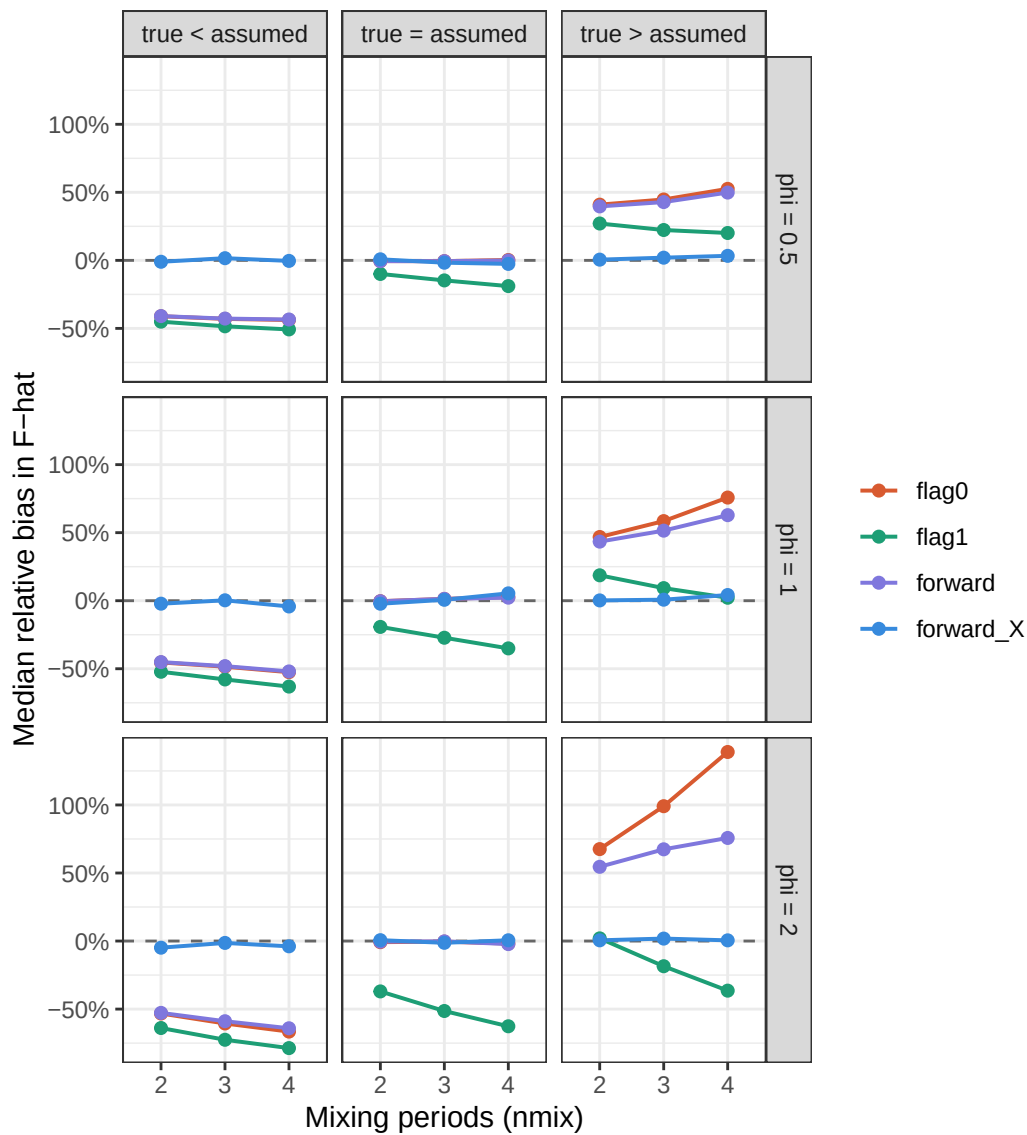


Figure 4: Median relative bias in F . Rows are the availability multiplier during mixing; columns are the reporting-rate scenario. In the middle column, where the reporting rate is correct, flag0 sits on zero in every cell; flag1 degrades steeply as either ϕ or $nmix$ increases.

```
knitr::kable(
  dcast(summ_b, nmix + phi + case ~ mode, value.var = "rel_bias"),
  digits = 3,
  caption = "Median relative bias in F-hat across the full design.")
```

Table 7: Median relative bias in F-hat across the full design.

nmix	phi	case	flag0	flag1	forward	forward_X
2	0.5	true < assumed	-0.411	-0.450	-0.410	-0.010
2	0.5	true = assumed	-0.005	-0.100	-0.006	0.008
2	0.5	true > assumed	0.409	0.271	0.397	0.005
2	1.0	true < assumed	-0.454	-0.522	-0.451	-0.022
2	1.0	true = assumed	-0.004	-0.193	-0.004	-0.022
2	1.0	true > assumed	0.468	0.187	0.434	0.002
2	2.0	true < assumed	-0.532	-0.639	-0.528	-0.049
2	2.0	true = assumed	-0.008	-0.370	-0.004	0.006
2	2.0	true > assumed	0.676	0.020	0.546	0.005
3	0.5	true < assumed	-0.430	-0.485	-0.428	0.016
3	0.5	true = assumed	-0.006	-0.147	-0.006	-0.017
3	0.5	true > assumed	0.447	0.223	0.428	0.020
3	1.0	true < assumed	-0.485	-0.579	-0.481	0.003
3	1.0	true = assumed	0.013	-0.272	0.013	0.007
3	1.0	true > assumed	0.585	0.093	0.515	0.007
3	2.0	true < assumed	-0.606	-0.725	-0.589	-0.014
3	2.0	true = assumed	-0.005	-0.514	-0.002	-0.013
3	2.0	true > assumed	0.991	-0.185	0.674	0.018
4	0.5	true < assumed	-0.438	-0.507	-0.435	-0.004
4	0.5	true = assumed	0.003	-0.189	-0.001	-0.026
4	0.5	true > assumed	0.526	0.201	0.498	0.033
4	1.0	true < assumed	-0.525	-0.631	-0.520	-0.042
4	1.0	true = assumed	0.027	-0.350	0.022	0.054
4	1.0	true > assumed	0.758	0.022	0.629	0.042
4	2.0	true < assumed	-0.665	-0.787	-0.641	-0.039
4	2.0	true = assumed	-0.018	-0.626	-0.022	0.006
4	2.0	true > assumed	1.389	-0.364	0.758	0.005

```
knitr::kable(
  dcast(summ_b, nmix + phi + case ~ mode, value.var = "mcse"),
  digits = 3,
  caption = paste0(
    "Monte Carlo standard error of the medians above. Differences ",
    "smaller than about twice these values should not be interpreted."))
```

Table 8: Monte Carlo standard error of the medians above. Differences smaller than about twice these values should not be interpreted.

nmix	phi	case	flag0	flag1	forward	forward_X
2	0.5	true < assumed	0.004	0.004	0.004	0.017
2	0.5	true = assumed	0.005	0.005	0.005	0.012
2	0.5	true > assumed	0.007	0.007	0.007	0.012
2	1.0	true < assumed	0.005	0.004	0.005	0.015
2	1.0	true = assumed	0.008	0.006	0.008	0.018
2	1.0	true > assumed	0.009	0.008	0.009	0.014
2	2.0	true < assumed	0.004	0.002	0.004	0.023
2	2.0	true = assumed	0.007	0.004	0.007	0.015
2	2.0	true > assumed	0.010	0.007	0.009	0.013
3	0.5	true < assumed	0.005	0.004	0.005	0.020
3	0.5	true = assumed	0.008	0.007	0.008	0.017
3	0.5	true > assumed	0.009	0.008	0.009	0.014
3	1.0	true < assumed	0.006	0.004	0.006	0.024
3	1.0	true = assumed	0.010	0.007	0.010	0.019
3	1.0	true > assumed	0.010	0.008	0.009	0.013
3	2.0	true < assumed	0.004	0.003	0.004	0.023
3	2.0	true = assumed	0.012	0.005	0.011	0.019
3	2.0	true > assumed	0.016	0.009	0.014	0.017
4	0.5	true < assumed	0.006	0.005	0.006	0.026
4	0.5	true = assumed	0.009	0.007	0.009	0.019
4	0.5	true > assumed	0.011	0.009	0.010	0.018
4	1.0	true < assumed	0.005	0.003	0.005	0.031
4	1.0	true = assumed	0.012	0.006	0.012	0.024
4	1.0	true > assumed	0.015	0.010	0.014	0.020
4	2.0	true < assumed	0.005	0.003	0.006	0.029
4	2.0	true = assumed	0.015	0.004	0.014	0.021
4	2.0	true > assumed	0.023	0.008	0.019	0.021

The survival floor engages in 0.0% of replicates across the 27-cell design. The Experiment B results can therefore be read as the floor-free case throughout, and Experiment C is where the floor becomes the story. Establishing this matters for how the two experiments are read against each other: the robustness result below is a floor-free result, and the Experiment C reversal is attributable to the floor rather than to any other difference between the designs.

4.2 The flag0 robustness result

One column of the design deserves separate attention: what happens when the reporting rate is assumed *correctly*, and only φ and `nmix` vary.

```
ggplot(summ_b[case == "true = assumed"],
       aes(factor(nmix), rel_bias, colour = mode, group = mode)) +
  geom_hline(yintercept = 0, linetype = 2, colour = "grey40") +
  geom_line(linewidth = 0.7) +
  geom_point(size = 1.8) +
  facet_wrap(~ phi, labeller = labeller(phi = function(x) paste0("phi = ", x))) +
  scale_y_continuous(labels = scales::percent) +
  scale_colour_manual(values = pal) +
  labs(x = "Mixing periods (nmix)", y = "Median relative bias in F-hat",
       colour = NULL)
```

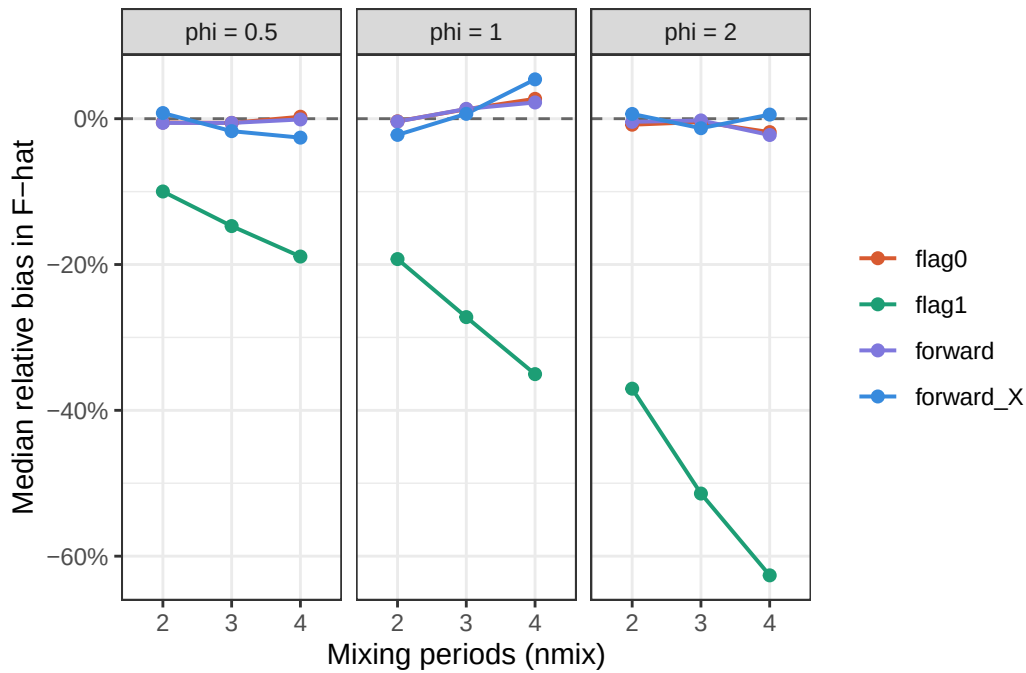


Figure 5: Relative bias when the assumed reporting rate is correct. `flag0` is essentially unbiased everywhere; `flag1`'s bias grows with both ϕ and mixing length.

`flag0` is close to unbiased across every combination of φ and `nmix`, provided the reporting rate is right — the largest absolute median bias in the nine cells is 2.7%. This is not an accident and it is worth stating plainly, because it is the strongest argument *for* the default setting: the

deterministic inversion solves for whatever mixing-period F reproduces the implied removal, so it never needs to know φ . Misspecifying the availability of tagged fish during mixing — the very thing the mixing period exists to handle — costs `flag0` nothing.

`flag1` has no such protection. Its error is the recaptures it fails to remove, which is a fraction $(1 - X)$ of the true recaptures, accumulated over every mixing period. Both raising φ and lengthening `nmix` increase the number of recaptures during mixing, so both make it worse — sharply. At $\varphi = 2$ and `nmix = 4` the bias in F is -63%, which is a large upward error in biomass.

4.3 Does bracketing ever occur?

```
wide_b <- bracket_check(
  res_b[, .(rep, nmix, phi, X_true, case, mode, F_hat, conv)], om$F_true)

brk_b <- wide_b[, .(pct = 100 * mean(brackets),
  width = median((hi - lo) / om$F_true)),
  by = .(nmix, phi, case)]

ggplot(brk_b, aes(factor(nmix), factor(phi), fill = pct)) +
  geom_tile(colour = "white", linewidth = 0.6) +
  geom_text(aes(label = sprintf("%.0f%%", pct)), size = 3.2) +
  facet_wrap(~ case) +
  scale_fill_gradient(low = "#F1EFE8", high = "#7F77DD", limits = c(0, 100)) +
  labs(x = "Mixing periods (nmix)", y = "phi", fill = "% bracketing")
```

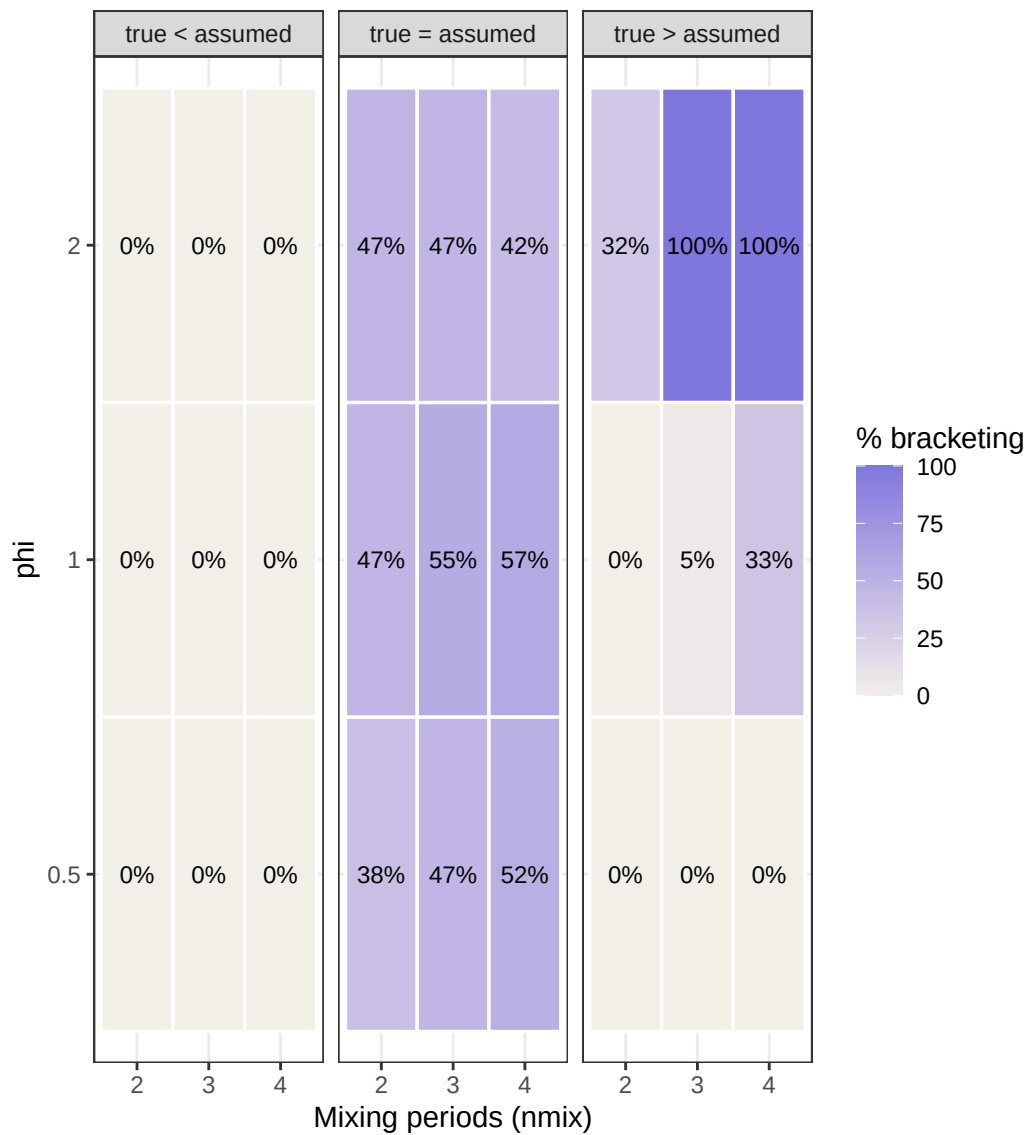


Figure 6: Percentage of replicates in which the true F falls between the two flag settings, by mixing length, tagged-fish availability, and reporting-rate scenario.

```
ggplot(brk_b, aes(factor(nmix), width, colour = factor(phi), group = factor(phi))) +
  geom_line(linewidth = 0.7) +
  geom_point(size = 1.8) +
  facet_wrap(~ case) +
  labs(x = "Mixing periods (nmix)", y = "Interval width / true F",
       colour = "phi")
```

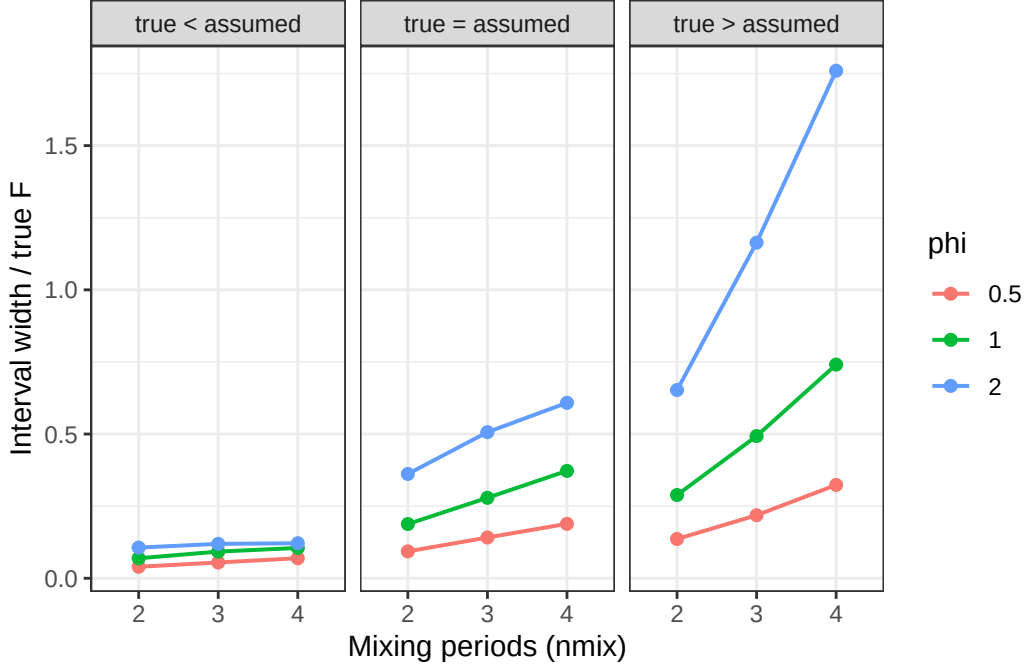


Figure 7: Width of the interval spanned by the two flag settings, as a multiple of the true F . Where bracketing does occur, the interval is extremely wide.

The heat map has a hard structural edge: **bracketing never occurs anywhere in the $\text{true} < \text{assumed}$ column** (maximum 0%), at any φ or nmix . That is not sampling noise, and it follows from the arithmetic. Because $X_{\text{assumed}} < 1$, the raised target R_t/X_{assumed} always exceeds R_t , so `flag0` always removes more tags than `flag1` and therefore always returns the *larger* $\hat{F}$. The interval is always $[\hat{F}_{\text{flag1}}, \hat{F}_{\text{flag0}}]$, with `flag0` as its upper end. Bracketing therefore requires $\hat{F}_{\text{flag0}} \geq F_{\text{true}}$, and `flag0` only sits above the truth when the true reporting rate is at or above what was assumed.

The pair is a one-sided construct. It can only ever bracket from below, and only when the assumed reporting rate errs low. If your concern is that the assumed reporting rate is too *high*, running both flag settings cannot bound the truth no matter what else is going on in the model — and no diagnostic internal to the pair will tell you which side you are on.

Within the two columns where bracketing does appear, it appears for different reasons:

- **true = assumed (38–57% throughout).** The degenerate case described in Experiment A: `flag0` is unbiased, so noise straddles the truth about half the time. The rate is flat across φ and nmix because it reflects Monte Carlo scatter, not a property of the configuration.

- **true > assumed (0% to 100%).** Genuine two-sided coverage, arising where `flag0` has blown up upward far enough that `flag1`'s own downward bias falls below the truth. This is two opposite-signed biases happening to cross. It appears in the following cells: `phi = 1, nmix = 3` (5%); `phi = 1, nmix = 4` (33%); `phi = 2, nmix = 2` (32%); `phi = 2, nmix = 3` (100%); `phi = 2, nmix = 4` (100%) — that is, it needs high availability, and at $\varphi = 1$ it needs a long mixing period as well.

Interval width tracks that story. Where bracketing is genuine, the interval is enormous: at $\varphi = 2$ and `nmix = 4` it spans roughly 1.8 times the true F . An interval that contains the truth but is nearly twice as wide as the quantity being estimated is not usable as an uncertainty statement. Where bracketing is degenerate (**true = assumed**), the interval is tight — 0.1 to 0.6 times F — but that tightness is meaningless, because it is anchored by an endpoint that happens to be unbiased for reasons the analyst cannot verify from inside the model.

4.4 The survival-floor condition

At extreme availability the raised target $R_t/X_{assumed}$ can drive the tagged population below the survival floor, which is the condition that triggers the `posfun` rescaling in `do_newton_raphson_for_tags()`.

It is worth being precise about the threshold, because the obvious statement of it is not the one the code implements. Reported returns can never exceed true recaptures, $R_t \leq C_t$, so *exhaustion* of the tagged population, $R_t/X_{assumed} > N_t$, requires $C_t/N_t > X_{assumed}$. But MULTIFAN-CL does not wait for exhaustion: it floors survival at $\tau = 0.22$ (the `age_flags(118)/(119)` defaults). The condition that actually engages the floor is therefore

$$\frac{R_t}{X_{assumed}} > (1 - \tau) N_t \quad \implies \quad \frac{C_t}{N_t} > (1 - \tau) X_{assumed},$$

which at the default τ is a catch fraction of about 0.78 times the assumed reporting rate — roughly a fifth easier to reach than the exhaustion version. At $X_{assumed} = 0.5$ that is 0.390; at $X_{assumed} = 0.2$ it is 0.156, which ordinary fishing mortalities reach without anything unusual happening.

This is the formal version of the trip-level argument: a trip taking a large part of a release group and reporting all of it engages the floor, and the lower the assumed reporting rate the smaller that part needs to be. The manual's warning is therefore less about reporting rates being uncertain than about their being low.

```
floor_grid <- CJ(nmix = c(2L, 3L, 4L), phi = c(0.5, 1.0, 2.0, 4.0),
                rep = seq_len(150), sorted = FALSE)
floor_grid[, cell := .GRP, by = .(nmix, phi)]
```

```

floor_res <- floor_grid[, {
  set.seed(1000L * cell + rep)
  r <- simulate_returns(om, X_true = 0.65, phi = phi, nmix = nmix)
  hits = floor_check_one(r, om, nmix))
}, by = .(rep, cell, nmix, phi)]

ggplot(floor_res[, .(pct = 100 * mean(hits > 0)), by = .(nmix, phi)],
  aes(factor(nmix), factor(phi), fill = pct)) +
  geom_tile(colour = "white", linewidth = 0.6) +
  geom_text(aes(label = sprintf("%.0f%%", pct)), size = 3.2) +
  scale_fill_gradient(low = "#F1EFE8", high = "#D85A30", limits = c(0, 100)) +
  labs(x = "Mixing periods (nmix)", y = "phi", fill = "% floored")

```

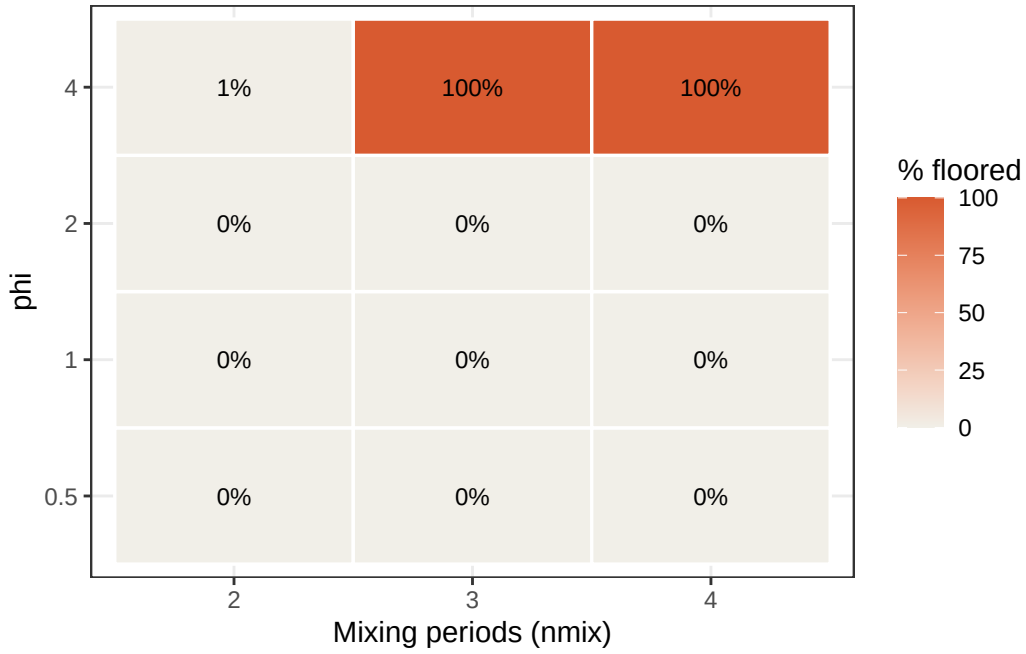


Figure 8: Frequency with which the flag0 removal drives survival below the floor, for the case where the true reporting rate exceeds the assumed value. The sweep extends past the main design to locate the boundary.

Within the range of φ used in the main design ($\varphi \leq 2$) the floor never engages. Beyond it, the floor is reached in more than half of replicates in the following cells: $\phi = 4$ with $\text{nmix} = 3$, $\phi = 4$ with $\text{nmix} = 4$.

The mechanism is worth noting because it is not obvious from the catch fraction alone: within a mixing period C_t/N_t is constant, but the *estimator's* tagged population declines faster than

the simulator's whenever the target is raised too far, so the ratio the floor tests against grows with each mixing period. That is why longer mixing periods trigger the floor at a given φ when short ones do not.

Two things are worth checking in any real configuration: whether that threshold is being reached for any release group, and whether `age_flags(117) < 0`, in which case the penalty weight is zero and the rescaling happens with no trace at all in the objective function.

A reader arriving here from Experiment C will see engagement rates that look incompatible with this sweep, and they are not. This sweep uses Experiment A/B's observation model: one release cohort of 10,000 tags with deterministic Baranov recaptures, in which the realised catch fraction barely departs from its expectation, so only extreme availability pushes it over the threshold. Experiment C uses several small groups with stochastic recaptures and clumped reporting, where the same *expected* catch fraction produces realisations well into the tail. The threshold is the same; what differs is how far the realisations stray from it, and that is a property of release-group size and the reporting process, not of the floor.

4.5 Recovery of the availability multiplier

```
phi_res <- res_b[mode %in% c("forward", "forward_X") & conv == 0 &
  case == "true > assumed",
  .(nmix, phi, mode, phi_hat)]

ggplot(phi_res, aes(factor(nmix), phi_hat, fill = mode)) +
  geom_boxplot(outlier.size = 0.4, alpha = 0.85) +
  geom_hline(aes(yintercept = phi), linetype = 2, colour = "grey40") +
  facet_wrap(~ phi, scales = "free_y",
    labeller = labeller(phi = function(x) paste0("true phi = ", x))) +
  scale_fill_manual(values = pal) +
  labs(x = "Mixing periods (nmix)", y = expression(hat(varphi)), fill = NULL)
```

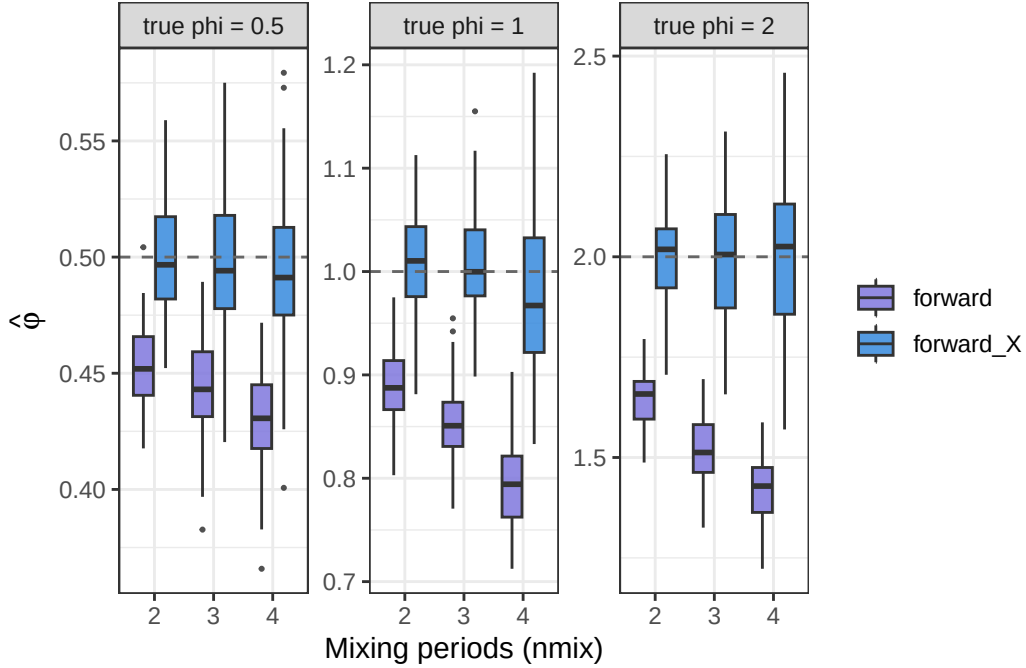


Figure 9: Estimated ϕ from the forward parameterisations, in the true $>$ assumed reporting-rate scenario. Panels are the true value of ϕ . Dashed lines are the true values.

Note that this figure conditions on one reporting-rate scenario (**true** $>$ **assumed**); the panels are values of φ , not reporting-rate cases.

forward_X recovers φ accurately at all three values, including $\varphi > 1$, and its accuracy is stable across mixing lengths — the largest median error across the nine (nmix , φ) combinations is 3%.

forward does not, and the pattern is diagnostic. With X pinned at the wrong value, $\hat{\varphi}$ is pulled toward 1 and the pull *strengthens* as the mixing period lengthens: at true $\varphi = 2$ the median falls from 1.66 at $\text{nmix} = 2$ to 1.43 at $\text{nmix} = 4$. The availability multiplier is absorbing the reporting-rate error, and a longer mixing period gives it more periods over which to do so. This is worth knowing before proposing φ to the developers: estimating it is only useful if the reporting rate is also free to move, otherwise φ becomes a sponge for reporting-rate misspecification rather than an estimate of availability.

4.6 Consistency with Experiment A

Experiment A’s configuration ($\text{nmix} = 2$, $\varphi = 0.5$) is one cell of Experiment B, fitted to independently drawn data. The two should agree to within Monte Carlo error, and checking that they do is the cheapest available test that the shared machinery is behaving.

```

ab <- merge(
  summ_a[, .(case, mode, A = rel_bias, A_mcse = mcse)],
  summ_b[nmix == 2L & phi == 0.5, .(case, mode, B = rel_bias, B_mcse = mcse)],
  by = c("case", "mode"))
ab[, `:=`(diff = A - B, joint_mcse = sqrt(A_mcse^2 + B_mcse^2))]

knitr::kable(ab[order(case, mode), .(case, mode, A, B, diff, joint_mcse)],
  digits = 3,
  caption = "Experiment A against the matching Experiment B cell.")

```

Table 9: Experiment A against the matching Experiment B cell.

case	mode	A	B	diff	joint_mcse
true < assumed	flag0	-0.414	-0.411	-0.003	0.005
true < assumed	flag1	-0.452	-0.450	-0.002	0.004
true < assumed	forward	-0.413	-0.410	-0.003	0.005
true < assumed	forward_X	-0.012	-0.010	-0.002	0.019
true = assumed	flag0	-0.004	-0.005	0.001	0.007
true = assumed	flag1	-0.098	-0.100	0.002	0.006
true = assumed	forward	-0.003	-0.006	0.003	0.007
true = assumed	forward_X	0.002	0.008	-0.006	0.015
true > assumed	flag0	0.409	0.409	-0.001	0.009
true > assumed	flag1	0.272	0.271	0.001	0.008
true > assumed	forward	0.396	0.397	-0.001	0.008
true > assumed	forward_X	0.000	0.005	-0.006	0.014

The deterministic and fixed- X estimators agree to within 0.3%. The largest discrepancy relative to its joint Monte Carlo error is forward in the true < assumed case, at 0.6 joint MCSEs. `forward_X` is the noisiest estimator throughout, so if any cell disagrees it should be that one — and a reader who notices two different numbers quoted for the same configuration in Experiments A and B is seeing sampling variability, not an inconsistency.

5 Experiment C: a realistic reporting process and the survival-floor failure mode

Experiments A and B use a deliberately clean observation model: a single release group of 10,000 tags, deterministic Baranov recaptures, and independent Poisson reporting. That model cannot produce the failure the MULTIFAN-CL manual warns about, because with 10,000 tags and independent reporting the realised recapture fraction never strays far from its expectation.

Three ingredients are missing, and all three are properties of real tagging data:

1. **Release groups are small.** Hundreds, not tens of thousands, once split by region and quarter.
2. **Recaptures are stochastic**, not a deterministic Baranov quantity.
3. **Reporting is clumped at the trip level.** Whether a tag is reported depends largely on whether *that trip* reports, so returns arrive in batches. The reporting rate is an average over trips, but the realisation for any one release group is close to all-or-nothing when few trips are involved.

This section adds all three and asks when the default setting actually breaks. The mixing-period machinery is unchanged from Experiments A and B — it is the same `mix_step` — so any difference in behaviour is attributable to the observation model and to release-group size, not to a different cap.

5.1 Machinery

```
# ---- stochastic Baranov + clumped trip-level reporting -----
# Each fish is caught, dies naturally, or survives (multinomial), so the
# population dynamics stay consistent with the Baranov equation the estimator
# assumes. Setting stochastic = FALSE and trip_size = 1 recovers the
# Experiment A/B observation model.
#
# `expected = TRUE` is a third path, added for the sufficiency test in
# Experiment D: it substitutes the expectation at BOTH stages, so neither the
# population dynamics nor the reporting step carries any Monte Carlo noise, and
# it works in doubles throughout because rounding the catch to whole fish would
# show up in the third decimal of a structural bias. `stochastic = FALSE` alone
# de-randomises only the dynamics -- the reporting step stays binomial -- which
# is why a separate argument is needed rather than an overload.
#
# The realised number of reporting trips per period is attached as an
# attribute, because it is the quantity that actually drives the clumping and
# it is not recoverable from trip_size alone.
simulate_group <- function(p, NO, X_g, phi, nmix,
                          trip_size = 1, stochastic = TRUE,
                          expected = FALSE) {
  if (expected && stochastic)
    stop("`expected` and `stochastic` are mutually exclusive; ",
         "set stochastic = FALSE when asking for expected returns.")
  if (expected && trip_size > 1)
    stop("`expected` is only defined for un-clumped reporting (trip_size = 1).")

  n <- if (expected) as.numeric(NO) else as.integer(NO)
```

```

r <- numeric(p$Tmax)
ktrips <- rep(NA_integer_, p$Tmax)

for (t in seq_len(p$Tmax)) {
  f_t <- p$F_true * if (t <= nmix) phi else 1
  z <- f_t + p$M
  s <- exp(-z)
  u <- f_t / z * (1 - s)          # caught
  v <- p$M / z * (1 - s)          # natural death

  if (stochastic) {
    draw <- rmultinom(1, n, c(u, v, s))
    catch <- draw[1]; surv <- draw[3]
  } else if (expected) {
    catch <- u * n; surv <- s * n
  } else {
    catch <- as.integer(round(u * n)); surv <- as.integer(round(s * n))
  }

  if (expected) {
    r[t] <- X_g * catch
    ktrips[t] <- NA_integer_
  } else if (catch > 0) {
    if (trip_size <= 1) {
      r[t] <- rbinom(1, catch, X_g)
      ktrips[t] <- catch
    } else {
      k <- max(round(catch / trip_size), 1)
      alloc <- rmultinom(1, catch, rep(1 / k, k))
      r[t] <- sum(alloc[runif(k) < X_g])
      ktrips[t] <- k
    }
  }
  n <- surv
}
attr(r, "ktrips") <- ktrips
r
}

# ---- multi-group objective, shared F -----
# `kernel = TRUE` replaces dpois() with the Poisson kernel  $\mu - r \log(\mu)$ . The
# discarded term,  $\lgamma(r + 1)$ , does not depend on any parameter, so the
# argmin is unchanged. It is needed because expected returns are not integers

```

```

# and dpois() returns zero density (and a warning) for non-integer x, which
# would make the deterministic fits in Experiment D fail rather than converge.
nll_multi <- function(par, groups, p, mode, nmix, prior_sd = 0.4,
                      kernel = FALSE) {
  f_pop <- exp(par[1])
  phi <- if (mode %in% c("forward", "forward_X")) exp(par[2]) else 1
  x <- if (mode == "forward_X") plogis(par[3]) else p$X_assumed

  nll <- 0
  for (g in seq_along(groups)) {
    r <- groups[[g]]$returns
    n <- groups[[g]]$NO

    for (t in seq_len(p$Tmax)) {
      premix <- t <= nmix

      if (premix && mode %in% c("flag0", "flag1")) {
        st <- mix_step(r[t], n, p, mode)
        f_t <- st$f
        nll <- nll + PENWT * st$pen
      } else {
        f_t <- if (premix) f_pop * phi else f_pop
        mu <- x * baranov(f_t, n, p$M) + 1e-10
        nll <- nll + if (kernel) mu - r[t] * log(mu)
                      else -dpois(r[t], mu, log = TRUE)
      }
      n <- n * exp(-(f_t + p$M))
    }
  }

  if (mode == "forward_X") {
    nll <- nll + 0.5 * ((log(x) - log(p$X_assumed)) / prior_sd)^2
  }
  nll
}

fit_multi <- function(groups, p, mode, nmix, interval = LOGF_INT,
                      kernel = FALSE) {
  mode <- as.character(mode)[1L]

  if (mode %in% c("flag0", "flag1")) {
    opt <- stats::optimize(
      function(lf) nll_multi(c(lf, 0, 0), groups, p, mode, nmix,

```

```

        kernel = kernel),
    interval = interval, tol = 1e-10)
return(data.table(F_hat = exp(opt$minimum), phi_hat = NA_real_,
                  X_hat = p$X_assumed, conv = 0L,
                  at_bound = at_bound_lf(opt$minimum, interval)))
}

opt <- stats::optim(
  par = c(log(0.10), 0, qlogis(p$X_assumed)),
  fn = function(par) nll_multi(par, groups, p, mode, nmix,
                              kernel = kernel),
  method = "Nelder-Mead",
  control = list(maxit = 4000, reltol = 1e-11))

data.table(F_hat = exp(opt$par[1]),
           phi_hat = exp(opt$par[2]),
           X_hat = if (mode == "forward_X") plogis(opt$par[3]) else p$X_assumed,
           conv = opt$convergence,
           at_bound = FALSE)
}

# ---- diagnostic: how often, and how widely, does the floor engage? -----
# Uses the same mix_step as the estimator, so the two advance the tagged
# population identically. If they were allowed to diverge, the floor frequency
# reported here would not be the one the estimator experienced.
floor_check <- function(groups, p, nmix) {
  hits <- 0L; tot <- 0L
  for (g in seq_along(groups)) {
    r <- groups[[g]]$returns; n <- groups[[g]]$NO
    for (t in seq_len(nmix)) {
      st <- mix_step(r[t], n, p, "flag0")
      if (st$floored) hits <- hits + 1L
      tot <- tot + 1L
      n <- n * exp(-(st$f + p$M))
    }
  }
  list(any = hits > 0L, hits = hits, frac = hits / tot)
}

```

5.2 Design

Eight release groups of 500 tags each, sharing one fishing mortality, with a mixing period of **nmix = 3 quarters** and availability $\varphi \in \{1, 2\}$. The mixing length is fixed here — Experiment B is the experiment that varies it — which matters when reading Experiment C against Experiment B's claim of robustness across **nmix**.

The assumed reporting rate becomes a factor: it is the axis the survival-floor condition depends on, and it is the quantity under dispute in the current assessment. Trip size is expressed as a percentage of the *release group*, so 25% means a trip large enough to take a quarter of the group.

Runtime note: this design carries all four estimators. Two of them are one-dimensional, so the cost is dominated by the two forward modes and is lower than the estimator count suggests.

Group-specific reporting rates are drawn on the logit scale and recentred there. Drawing them on the natural scale would require clamping away from zero, which at $X_{assumed} = 0.2$ truncates enough of the distribution to induce a small upward bias in the realised mean rate — an artefact of the sampling scheme that would be indistinguishable from an estimator property. The cost of working on the logit scale is that the arithmetic mean of the realised rates is correct only to within a Jensen term; that term is reported below so its size is visible rather than assumed.

```
n_rep_c    <- 30          # raise for smoother medians
G          <- 8           # release groups
NO_g       <- 500        # tags per group
rr_sd_lgt  <- 0.45       # between-group sd of logit(reporting rate)
NMIX_C     <- 3L         # mixing periods, fixed for this experiment

modes_c <- c("flag0", "flag1", "forward", "forward_X")

design_c <- CJ(X_assumed = c(0.20, 0.35, 0.50),
             trip_pct   = c(0.01, 0.10, 0.25),
             phi        = c(1.0, 2.0),
             mode       = modes_c,
             rep        = seq_len(n_rep_c), sorted = FALSE)

# `cell` deliberately excludes `mode`, so all four estimators see the same
# simulated dataset within a replicate and the comparison stays paired.
design_c[, cell := .GRP, by = .(X_assumed, trip_pct, phi)]

make_groups <- function(p, phi, nmix, trip_pct, rr_sd_logit) {
  lx <- rnorm(G, qlogis(p$X_assumed), rr_sd_logit)
  lx <- lx - mean(lx) + qlogis(p$X_assumed) # exact on the logit scale
  xs <- plogis(lx)
```

```

gs <- lapply(seq_len(G), function(g)
  list(NO = NO_g,
       X_g = xs[g],
       returns = simulate_group(p, NO_g, xs[g], phi, nmix,
                                trip_size = max(trip_pct * NO_g, 1),
                                stochastic = TRUE)))

gs
}

res_c <- design_c[, {
  p <- modifyList(om, list(X_assumed = X_assumed))
  set.seed(1000L * cell + rep)
  gs <- make_groups(p, phi, NMIX_C, trip_pct, rr_sd_lgt)
  fit <- fit_multi(gs, p, mode = mode, nmix = NMIX_C)
  fl <- floor_check(gs, p, NMIX_C)
  cbind(fit,
        floored      = fl$any,
        floor_frac    = fl$frac,
        mean_X        = mean(vapply(gs, `[`, numeric(1), "X_g")),
        min_ktrips     = min(vapply(gs, function(g) {
          kt <- attr(g$returns, "ktrips")[seq_len(NMIX_C)]
          kt <- kt[!is.na(kt)]
          if (!length(kt)) NA_real_ else as.numeric(min(kt))
        }, numeric(1)), na.rm = TRUE))
}, by = .(rep, cell, X_assumed, trip_pct, phi, mode)]

res_c[, `:=`(mode = factor(mode, levels = modes_c),
             rel_bias = F_hat / om$F_true - 1)]

knitr::kable(
  res_c[mode == "flag0",
        .(`mean realised X` = mean(mean_X),
          `relative error` = mean(mean_X) / X_assumed[1] - 1),
        by = .(`X assumed` = X_assumed)][order(`X assumed`)],
  digits = 4,
  caption = paste0(
    "Realised mean reporting rate against the assumed value. The residual ",
    "is the Jensen term from recentring on the logit scale."))

```

Table 10: Realised mean reporting rate against the assumed value. The residual is the Jensen term from recentring on the logit scale.

X assumed	mean realised X	relative error
0.20	0.2085	0.0425
0.35	0.3557	0.0162
0.50	0.5001	0.0001

```
trips_tab <- dcast(res_c[mode == "flag0",
                    .(median_trips = median(min_ktrips)),
                    by = .(phi, trip_pct)],
                  trip_pct ~ phi, value.var = "median_trips")
trips_tab[, trip_pct := 100 * trip_pct]
setnames(trips_tab, "trip_pct", "Trip size (%)")
```

```
knitr::kable(
  trips_tab, digits = 0,
  caption = paste0(
    "Median of the smallest number of reporting trips in any release group ",
    "during mixing, by phi (columns)."))
```

Table 11: Median of the smallest number of reporting trips in any release group during mixing, by phi (columns).

Trip size (%)	1	2
1	5	7
10	1	1
25	1	1

```
knitr::kable(
  dcast(res_c[, .(flagged = sum(conv != 0 | at_bound)), by = .(X_assumed, mode)],
        X_assumed ~ mode, value.var = "flagged"),
  caption = paste0("Experiment C: fits that either failed to converge or ",
    "returned an estimate on the edge of the search interval, ",
    "out of ", n_rep_c * 9, " per (X assumed, estimator)."))
```

Table 12: Experiment C: fits that either failed to converge or returned an estimate on the edge of the search interval, out of 270 per (X assumed, estimator).

X_assumed	flag0	flag1	forward	forward_X
0.20	0	0	0	0
0.35	0	0	0	0
0.50	0	0	0	0

The trip table is worth pausing on. At `trip_pct = 0.1` and $\varphi = 1$ the median release group has as few as 1 reporting trip in its worst mixing period. “A trip takes a quarter of the group” understates what is happening at that corner of the design: the period’s entire catch is close to a single all-or-nothing reporting decision. That is the cell driving the top-right of the map below, and it should be read as the extreme end of the clumping axis rather than as a typical fishery.

5.3 How often does the floor engage?

```
ro <- res_c[mode == "flag0" & conv == 0,
            .(pct = 100 * mean(floored), frac = 100 * mean(floor_frac)),
            by = .(X_assumed, trip_pct, phi)]

ggplot(ro, aes(factor(100 * trip_pct), factor(X_assumed), fill = pct)) +
  geom_tile(colour = "white", linewidth = 0.6) +
  geom_text(aes(label = sprintf("%.0f%%", pct)), size = 3.2) +
  facet_wrap(~ phi, labeller = labeller(phi = function(x) paste0("phi = ", x))) +
  scale_fill_gradient(low = "#F1EFE8", high = "#D85A30", limits = c(0, 100)) +
  labs(x = "Trip size (% of release group)", y = "Assumed reporting rate",
       fill = "% floored")
```

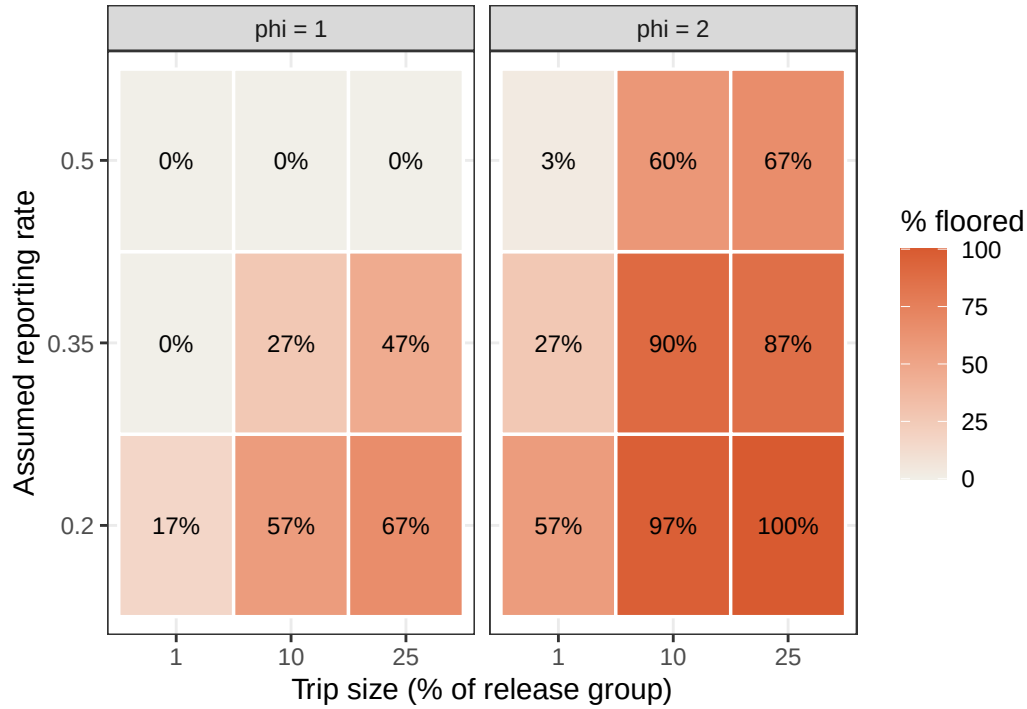


Figure 10: Percentage of replicates in which flag0's raised removal drives survival below the floor for at least one release group. The assumed reporting rate is correct on average on the logit scale in every cell; the small residual at the lowest assumed rate is the Jensen term reported above.

```
ggplot(ro, aes(factor(100 * trip_pct), factor(X_assumed), fill = frac)) +
  geom_tile(colour = "white", linewidth = 0.6) +
  geom_text(aes(label = sprintf("%.0f%%", frac)), size = 3.2) +
  facet_wrap(~ phi, labeller = labeller(phi = function(x) paste0("phi = ", x))) +
  scale_fill_gradient(low = "#F1EFE8", high = "#D85A30", limits = c(0, 100)) +
  labs(x = "Trip size (% of release group)", y = "Assumed reporting rate",
       fill = "% of group-periods")
```

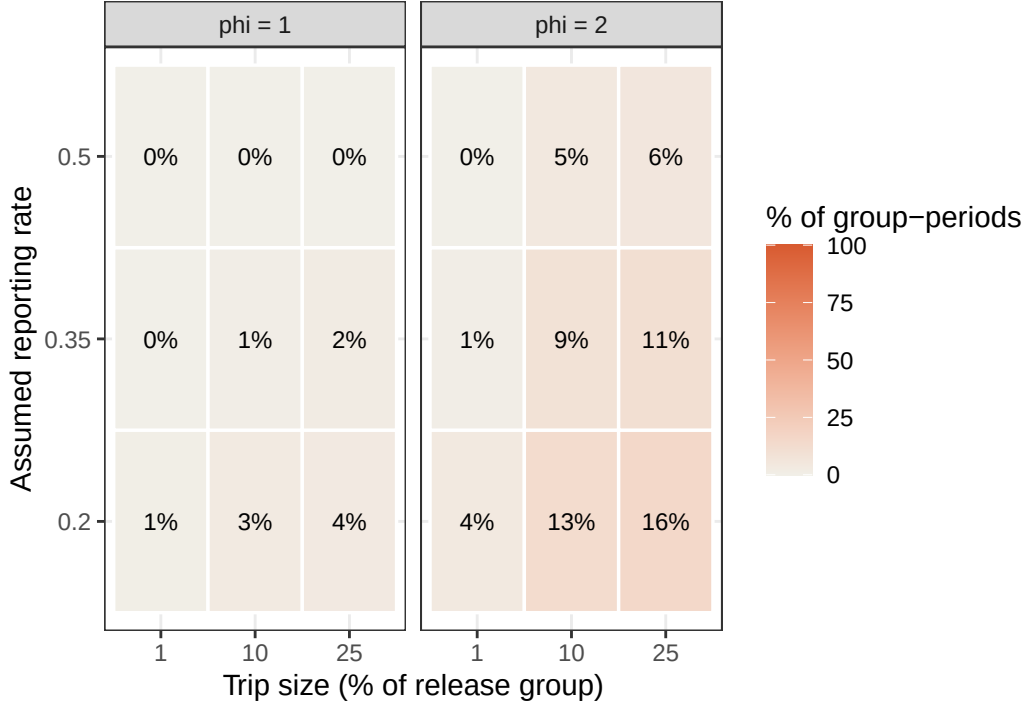


Figure 11: The same design, but showing the proportion of all group-periods floored rather than the proportion of replicates with at least one. A replicate in which one group trips the floor once is a different object from one in which most groups trip it repeatedly, and the binary map cannot distinguish them.

Three factors drive the failure, and none of them is a reporting-rate error.

The assumed reporting rate is the strongest. At $X_{assumed} = 0.5$ with $\varphi = 1$ the floor engages in 0% of replicates; at $X_{assumed} = 0.2$ with $\varphi = 2$ it is 57–100%. This follows directly from the condition above: a lower assumed rate sets a lower bar for the realised catch fraction to clear.

Availability during mixing matters nearly as much. Moving from $\varphi = 1$ to $\varphi = 2$ raises the $X_{assumed} = 0.5$ row from 0% to 3–67%. More recaptures during mixing means more opportunity for one of them to be an extreme realisation. This is why the summary qualifies the moderate-reporting-rate case: a moderate assumed rate is not on its own a guarantee that the floor is inactive.

Trip size matters, but it is the weakest of the three. Coarse reporting makes the failure more frequent — at $X_{assumed} = 0.2$, $\varphi = 1$ it rises from 17% to 67% across the trip-size range — but fine-grained reporting does *not* protect you when the other two factors are adverse: at $X_{assumed} = 0.2$ and $\varphi = 2$, trips taking just 1% of a release group still trigger it in 57% of replicates.

This is the mechanism behind the manual's recommendation, and it is governed mainly by the assumed reporting rate. An assessment that revises its tag-seeding estimates downward moves itself into the regime where the default breaks, without changing anything else about the model.

5.4 What it costs

```
ggplot(res_c[conv == 0], aes(factor(100 * trip_pct), rel_bias, fill = mode)) +  
  geom_hline(yintercept = 0, linetype = 2, colour = "grey40") +  
  geom_boxplot(outlier.size = 0.4, alpha = 0.85) +  
  facet_grid(X_assumed ~ phi,  
             labeller = labeller(phi = function(x) paste0("phi = ", x),  
                                 X_assumed = function(x) paste0("X = ", x))) +  
  scale_y_continuous(labels = scales::percent) +  
  scale_fill_manual(values = pal) +  
  labs(x = "Trip size (% of release group)", y = "Relative bias in F-hat",  
       fill = NULL)
```

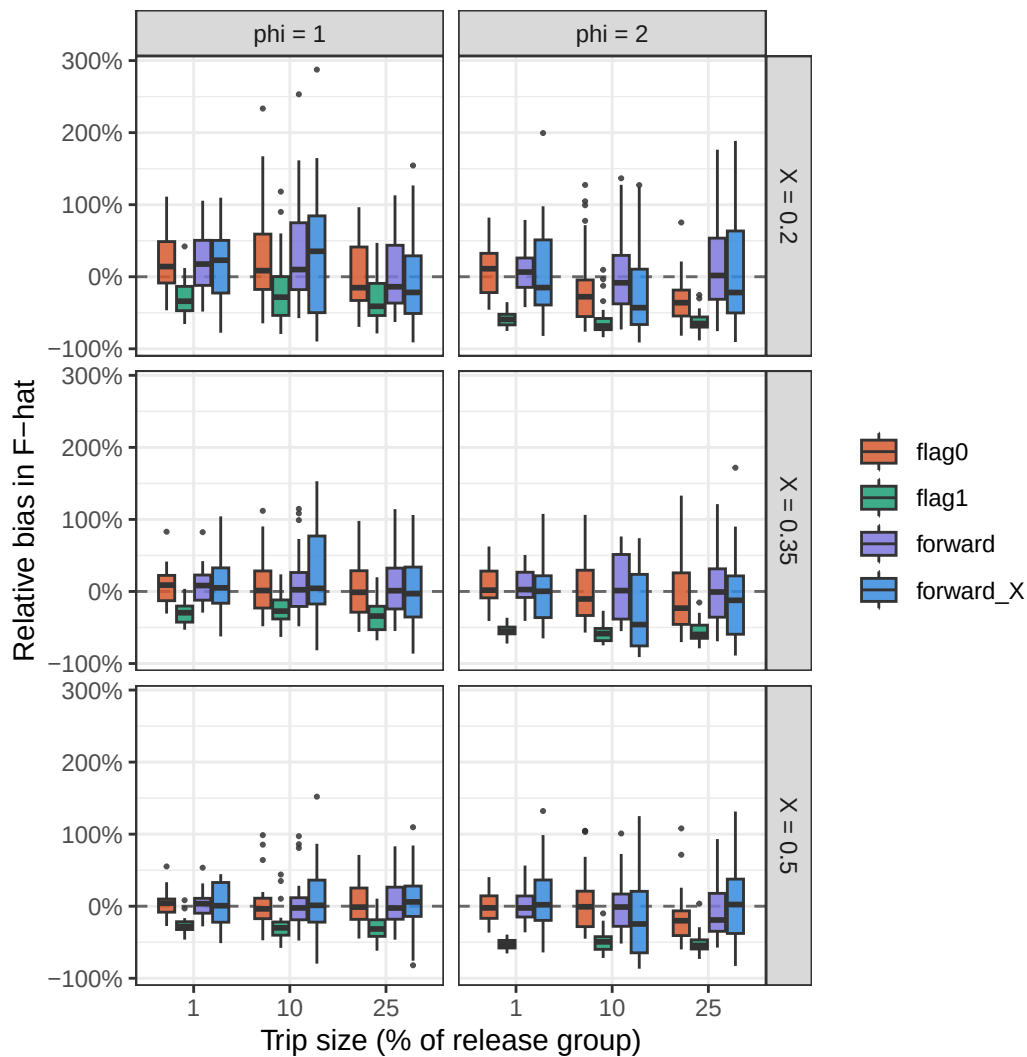


Figure 12: Distribution of relative bias in F . Where the floor rarely engages (X assumed = 0.5, $\phi = 1$) `flag0` stays centred with inflated spread; where it routinely engages (X assumed = 0.2, $\phi = 2$) `flag0`'s median turns substantially negative.

```
summ_c <- res_c[conv == 0,
  .(med = median(rel_bias),
    mcse = mcse_median(F_hat) / om$F_true,
    iqr = as.numeric(diff(quantile(rel_bias, c(.25, .75))))),
  by = .(X_assumed, trip_pct, phi, mode)]

# Median, IQR and MCSE go in three narrow tables rather than one wide one: at
# the width this document renders to, an eight-column dcast overprints its
```

```
# headers. Trip size is displayed as a percentage to match the prose.
disp_c <- copy(summ_c)[, trip_pct := 100 * trip_pct]
cols_c <- c("X assumed", "Trip size (%)", "phi", levels(disp_c$mode))

knitr::kable(
  dcast(disp_c, X_assumed + trip_pct + phi ~ mode, value.var = "med"),
  digits = 3, col.names = cols_c,
  caption = "Median relative bias in F-hat by estimator.")
```

Table 13: Median relative bias in F-hat by estimator.

X assumed	Trip size (%)	phi	flag0	flag1	forward	forward_X
0.20	1	1	0.141	-0.340	0.176	0.229
0.20	1	2	0.111	-0.592	0.065	-0.151
0.20	10	1	0.084	-0.283	0.100	0.353
0.20	10	2	-0.278	-0.684	-0.084	-0.429
0.20	25	1	-0.152	-0.410	-0.139	-0.218
0.20	25	2	-0.360	-0.649	0.017	-0.219
0.35	1	1	0.088	-0.295	0.082	0.051
0.35	1	2	0.018	-0.554	0.026	0.002
0.35	10	1	0.013	-0.272	0.024	0.044
0.35	10	2	-0.104	-0.585	0.012	-0.460
0.35	25	1	-0.012	-0.341	0.011	-0.031
0.35	25	2	-0.231	-0.597	-0.007	-0.123
0.50	1	1	0.042	-0.281	0.035	0.007
0.50	1	2	-0.023	-0.526	-0.024	0.020
0.50	10	1	-0.037	-0.300	-0.024	0.012
0.50	10	2	-0.008	-0.495	-0.012	-0.246
0.50	25	1	-0.012	-0.316	-0.025	0.060
0.50	25	2	-0.201	-0.545	-0.191	0.024

```
knitr::kable(
  dcast(disp_c, X_assumed + trip_pct + phi ~ mode, value.var = "iqr"),
  digits = 3, col.names = cols_c,
  caption = "Interquartile range of relative bias, same design.")
```

Table 14: Interquartile range of relative bias, same design.

X assumed	Trip size (%)	phi	flag0	flag1	forward	forward_X
0.20	1	1	0.575	0.338	0.625	0.730
0.20	1	2	0.544	0.147	0.408	0.904

X assumed	Trip size (%)	phi	flag0	flag1	forward	forward_X
0.20	10	1	0.768	0.539	0.928	1.343
0.20	10	2	0.508	0.155	0.672	0.768
0.20	25	1	0.742	0.446	0.802	0.800
0.20	25	2	0.360	0.141	0.849	1.138
0.35	1	1	0.352	0.223	0.349	0.490
0.35	1	2	0.372	0.094	0.350	0.581
0.35	10	1	0.515	0.264	0.471	0.944
0.35	10	2	0.628	0.170	0.897	0.993
0.35	25	1	0.575	0.325	0.567	0.693
0.35	25	2	0.715	0.183	0.671	0.809
0.50	1	1	0.183	0.108	0.206	0.551
0.50	1	2	0.312	0.106	0.292	0.561
0.50	10	1	0.284	0.183	0.306	0.583
0.50	10	2	0.492	0.176	0.448	0.853
0.50	25	1	0.435	0.230	0.445	0.421
0.50	25	2	0.343	0.127	0.527	0.754

```
knitr::kable(
  dcast(dispc, X_assumed + trip_pct + phi ~ mode, value.var = "mcse"),
  digits = 3, col.names = cols_c,
  caption = "Monte Carlo standard error of the medians above.")
```

Table 15: Monte Carlo standard error of the medians above.

X assumed	Trip size (%)	phi	flag0	flag1	forward	forward_X
0.20	1	1	0.092	0.055	0.089	0.118
0.20	1	2	0.078	0.024	0.073	0.144
0.20	10	1	0.159	0.108	0.172	0.214
0.20	10	2	0.128	0.052	0.136	0.151
0.20	25	1	0.114	0.079	0.119	0.155
0.20	25	2	0.075	0.033	0.149	0.179
0.35	1	1	0.058	0.035	0.057	0.099
0.35	1	2	0.061	0.021	0.057	0.098
0.35	10	1	0.091	0.048	0.099	0.147
0.35	10	2	0.104	0.030	0.101	0.126
0.35	25	1	0.097	0.052	0.096	0.106
0.35	25	2	0.125	0.033	0.128	0.142
0.50	1	1	0.041	0.027	0.040	0.068
0.50	1	2	0.048	0.017	0.052	0.095
0.50	10	1	0.077	0.054	0.081	0.119

X assumed	Trip size (%)	phi	flag0	flag1	forward	forward_X
0.50	10	2	0.099	0.033	0.089	0.126
0.50	25	1	0.071	0.043	0.073	0.099
0.50	25	2	0.083	0.034	0.081	0.117

The headline from Experiment B does **not** survive intact, and the pattern divides cleanly along the floor frequency mapped above.

Where the floor rarely engages, the cost is precision. At $X_{assumed} = 0.5$ with $\varphi = 1$, **flag0**’s median bias stays within 4%, consistent with Experiment B. But its interquartile range widens from 0.18 to 0.43 as trip size coarsens. Individual replicates swing in both directions, because a single release group whose trips happened to report can drive the raised removal, and hence the whole estimate of F , a long way.

Where the floor routinely engages, the cost is accuracy, and it runs downward. At $X_{assumed} = 0.2$ with $\varphi = 2$, **flag0**’s median bias ranges over -36% to +11%: +11% at trip size 1%, -28% at trip size 10%, -36% at trip size 25%. Between the least- and most-clumped cells the median moves by -47% against a joint Monte Carlo standard error of 10.8%, so that shift is resolved by this design. Taken step by step, 1 of the 2 successive differences is resolved against its own Monte Carlo error, so the step-by-step ordering of the cells is not a result of this design, and only the movement between its ends should be read as one. The cell on the positive side is the one where the floor bites least (4% of group-periods floored); the cell where it bites hardest (16%) is on the negative side. Note that even the mildest of the three floors in 57% of replicates, so engagement frequency alone does not locate the sign change. Floor severity and clumping rise together across this row, so the design cannot separate them – what it shows is that the sign turns over as the floor takes hold. The mechanism deserves to be stated explicitly: once **posfun** caps the removal, **flag0** can no longer take out the number of tags its own rule demands, so it **under-removes** — which is precisely **flag1**’s failure mode. Under floor engagement the default degenerates toward the alternative it is supposed to improve on, and the bias it contributes runs negative, meaning biomass biased *upward*.

This is the most important qualification to the Experiment B result. The unbiasedness of **flag0** is conditional not only on the reporting rate being right, but on the survival floor not engaging — and the second condition is easiest to violate exactly when assumed reporting rates are low.

flag1 behaves as Experiment D predicts throughout: median bias in -68% to -27%, driven by φ rather than by trip size, and consistently the most *precise* of the estimators (interquartile ranges of 0.09 to 0.54). Precisely wrong is still wrong, but **flag1**’s error is at least predictable in a way **flag0**’s is not.

forward_X needs a caveat of its own here. Its accuracy in Experiments A and B does not carry over to this small-sample, multi-group setting: medians range over -46% to +35%, and interquartile ranges reach 1.34. It is the widest of the estimators in 89% of the 18 cells. With

eight groups of 500 tags there is not enough information to estimate the reporting rate, the availability multiplier and F simultaneously. The likelihood-based treatment is structurally correct but it is not free: it needs data to support the extra parameters, and a fragmented tagging programme may not supply them.

5.5 Does the mixing-channel decomposition survive?

`forward` is included here for the same reason it exists in Experiments A and B: it handles the mixing period correctly while keeping X fixed at the assumed value, so the `flag0-forward` gap isolates what the mixing treatment contributes beyond the reporting-rate channel. Experiment C is the experiment that overturns the other Experiment B headlines, so the decomposition should not be assumed to carry over.

```
ggplot(summ_c[mode %in% c("flag0", "forward")],
       aes(factor(100 * trip_pct), med, colour = mode, group = mode)) +
  geom_hline(yintercept = 0, linetype = 2, colour = "grey40") +
  geom_line(linewidth = 0.7) +
  geom_point(size = 1.8) +
  facet_grid(X_assumed ~ phi,
            labeller = labeller(phi = function(x) paste0("phi = ", x),
                                X_assumed = function(x) paste0("X = ", x))) +
  scale_y_continuous(labels = scales::percent) +
  scale_colour_manual(values = pal) +
  labs(x = "Trip size (% of release group)", y = "Median relative bias in F-hat",
       colour = NULL)
```

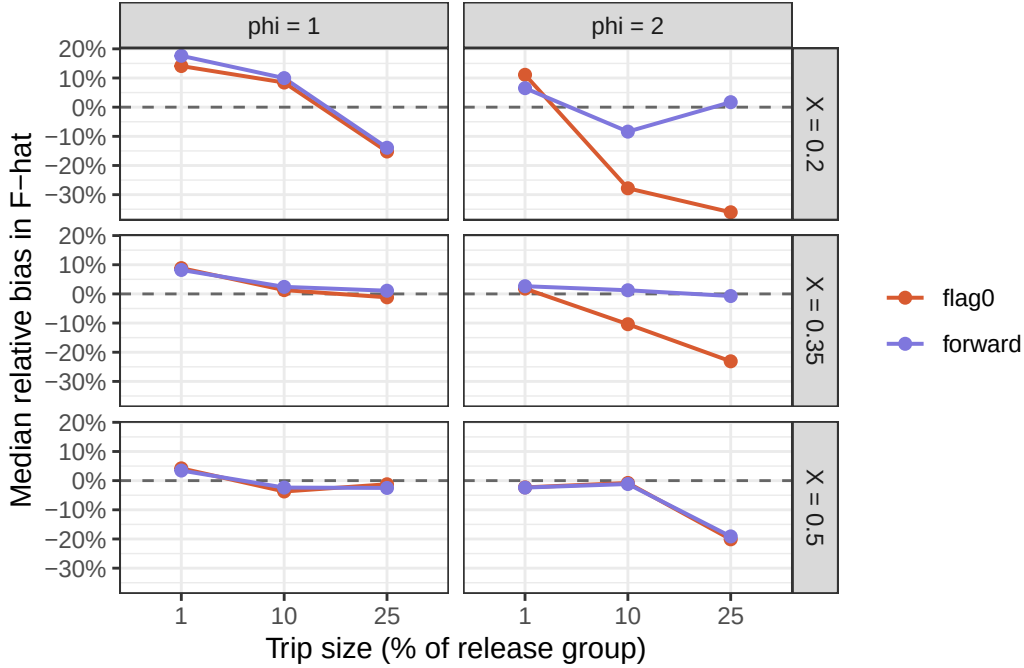


Figure 13: Median bias for `flag0` and `forward` in Experiment C. The vertical distance between the two series is the contribution of the deterministic mixing treatment, holding the reporting-rate treatment fixed.

Note that in this experiment the reporting rate is correct on average, so `forward` is not carrying a reporting-rate error in the way it does in Experiments A and B; the gap here isolates the deterministic inversion against a correctly specified alternative rather than decomposing a misspecification. Where the floor is inactive ($X_{assumed} = 0.5$, $\varphi = 1$) the two agree to within 1%. Where it is active the gap widens to -38% to +5%, with the largest single gap (-38%) at $X_{assumed} = 0.2$, $\varphi = 2$, trip size 25%. The deterministic treatment is therefore a second-order channel exactly where the floor is inactive and a first-order one where it is not — the same boundary that governs everything else in this experiment.

6 Experiment D: the mixing-period recapture fraction

Experiment B varied φ and `nmix` separately, which is how an analyst encounters them — one is a property of the fishery, the other a configuration choice. But for `flag1` they are not two effects. Both act only by changing how many tags are recaptured while the mixing rule is in force.

Define

$$\psi = \frac{1}{N_0} \sum_{t \leq n_{mix}} C_t,$$

the expected fraction of a release group recaptured during the mixing period. This is directly computable from a real `.tag` file given `tag_flags(i,1)`, which makes it the useful axis: an analyst can measure ψ for their own data and read off the expected bias.

6.1 Why ψ should govern the `flag1` bias

`flag1` removes R_t where it should remove C_t , so in expectation it under-removes by $(1 - X) C_t$ each mixing period, and $(1 - X) \psi N_0$ in total. The surviving tagged cohort entering the informative periods is therefore too large by that amount. Predicted recaptures scale with the cohort, so to match the observed post-mixing returns the model compensates with a smaller F :

$$\frac{\hat{F}}{F} \approx \frac{N_{post}}{N_{post} + (1 - X) \psi N_0}, \quad N_{post} = N_0 e^{-(\varphi F + M) n_{mix}}.$$

Note carefully what this does and does not predict. The bias depends on φ and `nmix` only through ψ and N_{post} — but N_{post} carries an `nmix` dependence of its own, over and above the one that runs through ψ . Two configurations with different φ and `nmix` but the same ψ are therefore predicted to be *close*, not identical, and the one with the longer mixing period is predicted to be the worse of the two: it reaches the same recapture fraction with a smaller surviving cohort to carry the un-removed tags. Whether that separation is large enough to matter for an assessment is a quantitative question, and it is the one the sufficiency test below is built to answer.

```
psi_of <- function(p, phi, nmix) {
  n <- p$N0; tot <- 0
  for (t in seq_len(nmix)) {
    tot <- tot + baranov(p$F_true * phi, n, p$M)
    n <- n * exp(-(p$F_true * phi + p$M))
  }
  tot / p$N0
}

bias_pred <- function(p, phi, nmix, X) {
  psi <- psi_of(p, phi, nmix)
  npost <- p$N0 * exp(-(p$F_true * phi + p$M) * nmix)
  under <- (1 - X) * psi * p$N0
  -under / (npost + under)
```

```

}

# psi_of() is monotone increasing in phi at fixed nmix, so it can be inverted
# for a target recapture fraction. This is what turns psi from an outcome of
# the design into a controlled variable, and it is the whole basis of the
# sufficiency test below: without it, no two cells share a psi exactly.
phi_for_psi <- function(target, nmix, p = om, phi_max = 20) {
  if (psi_of(p, phi_max, nmix) < target) return(NA_real_)
  uniroot(function(ph) psi_of(p, ph, nmix) - target,
          interval = c(1e-8, phi_max), tol = 1e-10)$root
}

# The loop in psi_of() and the closed form below are the same quantity: summing
# Baranov catch over a geometrically declining cohort telescopes to
# (phi*F/Z)(1 - exp(-nmix*Z)). Asserting it keeps the two from drifting if
# either is edited, and documents where the closed form comes from -- it is the
# expression an analyst needs to compute expected psi for a configuration they
# have not run.
psi_closed <- function(p, phi, nmix) {
  z <- p$F_true * phi + p$M
  (p$F_true * phi / z) * (1 - exp(-nmix * z))
}
stopifnot(all(abs(mapply(function(ph, nm) psi_of(om, ph, nm) -
                        psi_closed(om, ph, nm),
                        c(0.25, 0.5, 1, 2, 3),
                        c(2L, 3L, 4L, 2L, 3L))) < 1e-12))

```

6.2 Simulation

Reporting is left un-clumped here so that ψ is isolated from the dispersion effects of Experiment C. Four release groups of 2,000 tags, correct reporting rate throughout, so the *only* thing varying is how much of each group is recaptured before the mixing rule releases its grip.

```

n_rep_d <- 40
NO_d    <- 2000
G_d     <- 4

design_d <- CJ(phi = c(0.25, 0.5, 1.0, 2.0, 3.0),
              nmix = c(2L, 3L, 4L),
              mode = c("flag0", "flag1"),
              rep  = seq_len(n_rep_d), sorted = FALSE)
design_d[, cell := .GRP, by = .(phi, nmix)]

```

```

res_d <- design_d[, {
  set.seed(1000L * cell + rep)
  gs <- lapply(seq_len(G_d), function(g)
    list(NO = NO_d,
         returns = simulate_group(om, NO_d, om$X_assumed, phi, nmix,
                                   trip_size = 1, stochastic = TRUE)))
  fit_multi(gs, om, mode = mode, nmix = nmix)
}, by = .(rep, cell, phi, nmix, mode)]

res_d[, `:=`(mode = factor(mode, levels = c("flag0", "flag1")),
          rel_bias = F_hat / om$F_true - 1)]

summ_d <- res_d[conv == 0, .(bias = median(rel_bias),
                                     mcse = mcse_median(F_hat) / om$F_true),
               by = .(phi, nmix, mode)]
summ_d[, psi := psi_of(om, phi, nmix), by = .(phi, nmix)]
summ_d[, pred := bias_pred(om, phi, nmix, om$X_assumed), by = .(phi, nmix)]

ggplot(summ_d, aes(psi, bias)) +
  geom_hline(yintercept = 0, linetype = 2, colour = "grey40") +
  geom_line(data = summ_d[mode == "flag1"][order(psi)],
            aes(y = pred), linetype = 2, colour = "grey30") +
  geom_linerange(aes(ymin = bias - 2 * mcse, ymax = bias + 2 * mcse,
                    colour = mode), alpha = 0.6) +
  geom_point(aes(colour = mode, shape = factor(nmix)), size = 2.4) +
  scale_y_continuous(labels = scales::percent) +
  scale_colour_manual(values = pal) +
  labs(x = expression(psi~"(fraction of release group recaptured during mixing)"),
       y = "Median relative bias in F-hat",
       colour = NULL, shape = "nmix")

```

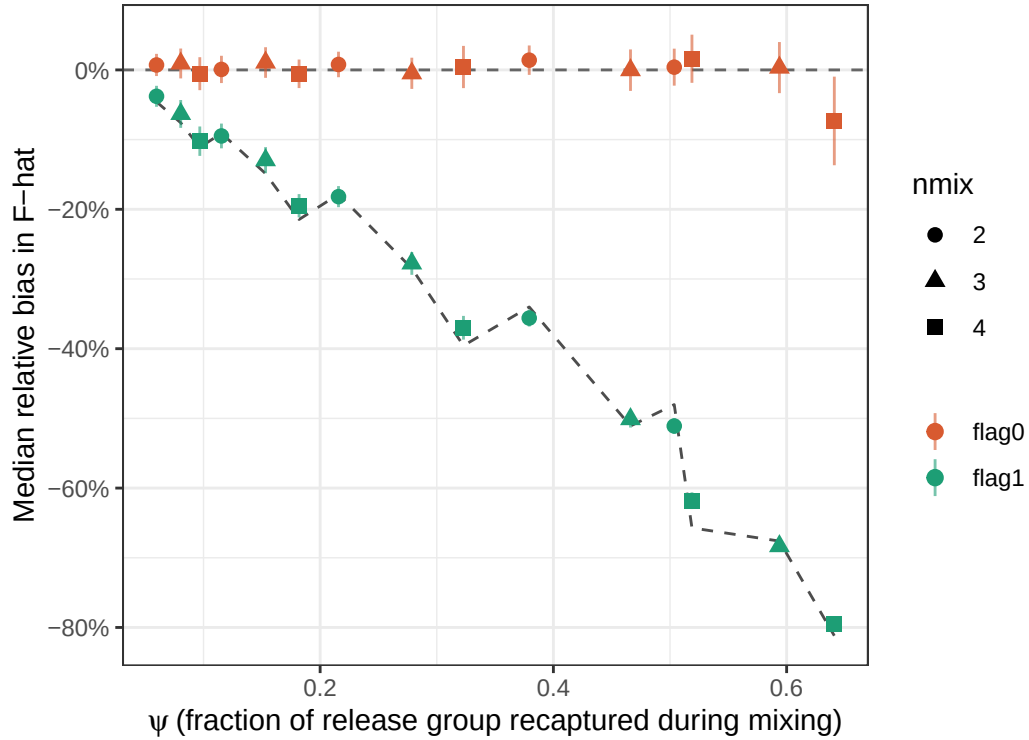


Figure 14: Median relative bias against the mixing-period recapture fraction. Points are simulated, with error bars at two Monte Carlo standard errors; the dashed line is the analytic prediction for flag1. Configurations with different ϕ and n_{mix} collapse onto a single curve.

```
tab_d <- merge(dcast(summ_d, phi + nmix + psi ~ mode, value.var = "bias"),
               summ_d[mode == "flag1", .(phi, nmix, predicted = pred)],
               by = c("phi", "nmix"))

knitr::kable(tab_d[order(psi), .(phi, nmix, psi, flag0, flag1, predicted)],
              digits = 3,
              col.names = c("phi", "nmix", "psi", "flag0", "flag1",
                           "flag1 predicted"),
              caption = paste0(
                "Bias against the mixing-period recapture fraction, with the ",
                "analytic prediction."))
```

Table 16: Bias against the mixing-period recapture fraction, with the analytic prediction.

phi	nmix	psi	flag0	flag1	flag1 predicted
0.25	2	0.060	0.007	-0.038	-0.046
0.25	3	0.080	0.009	-0.063	-0.076
0.25	4	0.097	-0.005	-0.102	-0.111
0.50	2	0.115	0.001	-0.095	-0.091
0.50	3	0.153	0.011	-0.130	-0.149
0.50	4	0.182	-0.006	-0.195	-0.215
1.00	2	0.216	0.008	-0.182	-0.178
1.00	3	0.279	-0.005	-0.278	-0.285
1.00	4	0.323	0.004	-0.370	-0.396
2.00	2	0.379	0.014	-0.356	-0.340
2.00	3	0.466	0.000	-0.501	-0.511
3.00	2	0.504	0.004	-0.511	-0.480
2.00	4	0.519	0.016	-0.618	-0.657
3.00	3	0.594	0.003	-0.683	-0.676
3.00	4	0.641	-0.073	-0.796	-0.812

```
d_cor <- summ_d[mode == "flag1", cor(bias, pred)]
cat(sprintf("Correlation between simulated and predicted flag1 bias: %.4f\n",
           d_cor))
```

Correlation between simulated and predicted flag1 bias: 0.9975

6.3 Is ψ sufficient?

The design above crosses φ and nmix and reports whatever ψ falls out, so no two cells share a ψ exactly and the collapse onto a single curve can only be judged by eye against the fitted line. The question the prediction actually raises — whether two configurations with the *same* ψ behave the same — needs a design in which ψ is controlled rather than observed.

Inverting `psi_of()` supplies one. Solving for the φ that delivers a target ψ at each mixing length puts the cells on a common ψ grid, and fitting to expected rather than simulated returns removes the Monte Carlo noise entirely, so any separation between mixing lengths at the same ψ is structural rather than scatter. No Monte Carlo standard errors are reported in this subsection because there is no Monte Carlo.

```
PSI_TARGETS <- c(0.05, 0.10, 0.20, 0.30, 0.40, 0.50)
```

```
calib <- CJ(psi_t = PSI_TARGETS, nmix = 2:4)
calib[, phi := mapply(phi_for_psi, psi_t, nmix)]
```

```
calib <- calib[is.finite(phi)]

knitr::kable(dcast(calib, psi_t ~ nmix, value.var = "phi"), digits = 2,
  col.names = c("psi", "nmix = 2", "nmix = 3", "nmix = 4"),
  caption = paste0(
    "Availability multiplier required to reach each target ",
    "recapture fraction, by mixing length."))
```

Table 17: Availability multiplier required to reach each target recapture fraction, by mixing length.

psi	nmix = 2	nmix = 3	nmix = 4
0.05	0.21	0.15	0.13
0.10	0.43	0.31	0.26
0.20	0.92	0.68	0.56
0.30	1.48	1.10	0.91
0.40	2.15	1.60	1.34
0.50	2.97	2.23	1.88

This calibration table is worth reading before the results. Reaching a high ψ over a short mixing period requires a large availability multiplier; reaching the same ψ over four periods requires much less. The top two rows need multipliers above the range Experiment B covered, so they complete the grid rather than describing configurations anyone should expect to meet.

```
grid_suff <- CJ(psi_t = PSI_TARGETS, nmix = 2:4,
  mode = c("flag0", "flag1"), sorted = FALSE)
grid_suff <- merge(grid_suff, calib, by = c("psi_t", "nmix"))

res_suff <- grid_suff[, {
  # One release group, expected returns, correct reporting rate. The Poisson
  # kernel rather than dpois(), because expected returns are not integers.
  g <- list(list(NO = om$NO,
    returns = simulate_group(om, om$NO, om$X_assumed, phi, nmix,
      trip_size = 1, stochastic = FALSE,
      expected = TRUE)))
  cbind(fit_multi(g, om, mode = mode, nmix = nmix, kernel = TRUE),
    floored = floor_check(g, om, nmix)$any)
}, by = .(psi_t, nmix, phi, mode)]

res_suff[, `:=`(rel_bias = F_hat / om$F_true - 1,
  psi = mapply(function(ph, nm) psi_of(om, ph, nm),
    phi, nmix))]
```

```

res_suff[, pred := mapply(function(ph, nm) bias_pred(om, ph, nm, om$X_assumed),
                           phi, nmix)]

# The inversion should be exact, and nothing here should be sitting on a search
# bound or against the survival floor -- if either happens the numbers below
# describe the machinery rather than the estimator.
stopifnot(max(abs(res_suff$psi - res_suff$psi_t)) < 1e-8,
           !any(res_suff$at_bound))

suff_f1 <- res_suff[mode == "flag1"]
suff_f0 <- res_suff[mode == "flag0"]

knitr::kable(dcast(suff_f1, psi_t ~ nmix, value.var = "rel_bias"), digits = 3,
              col.names = c("psi", "nmix = 2", "nmix = 3", "nmix = 4"),
              caption = paste0(
                "Structural relative bias in F-hat under tag_flags(i,2) = 1, ",
                "at a controlled recapture fraction (rows) and mixing length ",
                "(columns), with the reporting rate correctly specified at ",
                fnum(om$X_assumed, 2), ". Monte Carlo noise is absent by ",
                "construction."))

```

Table 18: Structural relative bias in $\hat{F}$ under $\text{tag_flags}(i,2) = 1$, at a controlled recapture fraction (rows) and mixing length (columns), with the reporting rate correctly specified at 0.50. Monte Carlo noise is absent by construction.

psi	nmix = 2	nmix = 3	nmix = 4
0.05	-0.039	-0.044	-0.048
0.10	-0.081	-0.090	-0.100
0.20	-0.174	-0.192	-0.211
0.30	-0.277	-0.305	-0.334
0.40	-0.389	-0.427	-0.464
0.50	-0.509	-0.555	-0.599

ψ is close to sufficient, but it is not sufficient. At every target the ordering the theory predicts is present — a longer mixing period carries the worse bias at the same recapture fraction — and the separation widens as ψ does, from 0.9% at $\psi = 0.05$ to 9.1% at $\psi = 0.50$. Against a bias of -59.9% in that worst cell, a spread of that size is second order but not negligible: an analyst reading a single number off a ψ curve is making an error of roughly 15% of the quantity they are estimating, in a known direction.

That the separation exists is not a failure of the theory in Section 6.1 — the predictor carries the same `nmix` dependence through N_{post} and reproduces the ordering, giving a spread of 15.6%

at the top of the grid against the fitted 9.1%. The approximation overstates the effect while getting its sign and ordering right. What fails is the looser statement, easy to make from the fitted curve alone, that the bias is a function of ψ *simpliciter*. It is a function of ψ and the mixing length, with the second term small.

flag0 is flat across the whole grid, as it must be: with expected returns and a correct reporting rate its inversion is exact. Its largest absolute deviation anywhere in the design is 0.0000%, which is the optimiser's tolerance rather than a property of the estimator, and it is worth contrasting with the 7.3% residual in the Monte Carlo design above: that residual is sampling scatter, not structure, and this is the run that demonstrates it. The survival floor did not engage anywhere in this design, so the **flag0** row is a clean benchmark and the **flag1** numbers are attributable to the mixing-period treatment alone.

6.4 Reading this for a real assessment

The near-collapse onto a single curve is the practical result. **flag1**'s bias is not a property of the mixing-period length or of tag availability separately — it is largely a function of the recapture fraction those two things produce, and that fraction is measurable. For a real release group:

1. Take **tag_flags(i,1)** for that group and sum the *reported returns* within that many periods of release.
2. Raise by the assumed reporting rate and divide by the number released: $\hat{\psi}_i \approx (1/X) \sum_{t \leq \text{tag_flags}(i,1)} R_{it}/N_i$. The **.tag** file records returns, not recaptures, so omitting the $1/X$ returns $X\psi$ — at a reporting rate near 0.5, half the true fraction.
3. Read the expected downward bias in F off the curve, or evaluate $-(1-X)\psi N_0/(N_{post} + (1-X)\psi N_0)$ directly. Where the mixing period is long, take the pessimistic end of the range at that ψ .

Two caveats travel with $\hat{\psi}$. The raising uses the same assumed reporting rate whose correctness is in question, so $\hat{\psi}$ inherits that error — and inherits it in the unhelpful direction, since a reporting rate that is too high understates ψ and therefore understates the bias. And a release-group-weighted mean aggregates over groups whose mixing lengths and availabilities differ, whereas the figures here apply a single value to a single cohort.

At $\psi = 0.05$ the bias is -4.8% to -3.9% and arguably tolerable. By $\psi = 0.3$ it is -33.4% to -27.7%, and by $\psi = 0.50$ it reaches -59.9% to -50.9%. Whether **tag_flags(i,2) = 1** is defensible for a given assessment is therefore an empirical question about that assessment's tag data, not a general one — and it is a question the assessment team can answer in an afternoon.

flag0 is far flatter across the same range, but it is not exactly zero: its largest absolute median bias is -7% at $\psi = 0.64$, and 1 of the 15 cells sits more than two Monte Carlo standard errors from zero. That single exception carries a negative sign and sits at the high- ψ end of the design. Whether that residual is Monte Carlo scatter or a real small-sample property of inverting a

noisy target is what Experiment E asks. Note the direction the theory predicts: inverting a noisy target is *convex* in the target, so the mechanism, if present, biases F **upward** at small samples.

7 Experiment E: release-group size and the noisy-inversion bias

The `flag0` residuals at high ψ have a candidate explanation. The estimator inverts an *observed* quantity: it solves for the F that removes $R_t/X_{assumed}$ tags. That inversion is a convex function of the target over the relevant range, so by Jensen’s inequality the expectation of the solved F exceeds the F that would be solved from the expected target. The size of the gap grows with the noise-to-signal ratio of R_t , which falls with release-group size.

If that is the mechanism, `flag0`’s bias should decline systematically as N_0 rises, holding ψ fixed. It would matter if it did: it would be a *third* condition on the default’s unbiasedness — adequate release-group size — independent of both the reporting rate and the survival floor, and small release groups are the realistic case once releases are split by region and quarter.

Three design points are worth stating before the result, because each of them determines whether the experiment can detect the effect it is looking for and whether the number it reports means what it appears to mean.

The statistic has to be tail-aware, and the mean alone is not enough. Jensen’s inequality is a statement about $E[\hat{F}]$, so the median — close to invariant under the monotone convexity being hypothesised — tests the mechanism with an instrument largely blind to it. But the mean has a problem of its own here. The solved F is unbounded above in principle and bounded in practice only by the search interval, so if the effect is carried by a handful of extreme replicates, the mean is partly a report on where the interval was placed. Three statistics are therefore reported together: the **frequency of gross overshoot** (the proportion of replicates returning $\hat{F}$ more than $5\times$ the truth), which is the primary statistic because it is a bounded frequency and insensitive to the interval as long as the ceiling sits well above the threshold; the mean, which is what the convexity argument speaks to; and the median, for comparability with the rest of this document.

The whole design is run at two search intervals. Every cell is fitted twice, once over the interval used everywhere else in this document and once over a much wider one. If the reported effect is a property of the estimator, the two runs agree. If it is a property of the ceiling, they do not, and the mean is then not a quantity worth quoting. This check is reported before the verdict, not after it.

One release group, not four. `fit_multi` pools groups into a single objective, so four groups over four mixing periods averages sixteen independent noisy inversions before $\hat{F}$ is formed. That averaging is exactly what the large-sample argument says washes the bias out, so pooling suppresses the quantity of interest. Experiment E therefore uses a single group, which maximises the noise-to-signal ratio available at a given N_0 .

Recapture fraction is held high — $\psi = 0.52$, toward the upper end of the Experiment D range — because that is where the Experiment D residuals appeared. Total tags vary with group size, which affects the precision of the estimates but not their expectation; the MCSE columns are what to read against.

Runtime note: fitting at two intervals doubles this design, and the replicate count is set high because a tail frequency needs more replicates than a mean does. Every fit is still one-dimensional, so this remains among the cheaper of the five experiments; drop `n_rep_e` for a quick pass, but check the guard margin reported at the end of the section before trusting the verdict.

```
# A tail frequency is a much lower-powered statistic than a mean, so this
# experiment needs more replicates than a median-based one would. At the tail
# mass seen here, 200 replicates leaves the difference between the smallest and
# largest group sizes barely separable from its own Monte Carlo error.
n_rep_e <- 400
PHI_E    <- 2.0
NMIX_E    <- 4L
G_e      <- 1          # one group: pooling averages away the effect being tested

# The same simulated data are fitted over both intervals, so the two runs are
# paired and any difference is attributable to the ceiling alone.
intervals_e <- list(default = LOGF_INT, wide = LOGF_INT_WIDE)

design_e <- CJ(NO_e      = c(125, 250, 500, 2000, 10000),
              interval = names(intervals_e),
              rep       = seq_len(n_rep_e), sorted = FALSE)
design_e[, cell := .GRP, by = .(NO_e)]

res_e <- design_e[, {
  set.seed(1000L * cell + rep)
  gs <- lapply(seq_len(G_e), function(g)
    list(NO = NO_e,
         returns = simulate_group(om, NO_e, om$X_assumed, PHI_E, NMIX_E,
                                   trip_size = 1, stochastic = TRUE)))
  fit_multi(gs, om, mode = "flag0", nmix = NMIX_E,
            interval = intervals_e[[interval]])
}, by = .(rep, cell, NO_e, interval)]

res_e[, `:=`(rel_bias = F_hat / om$F_true - 1,
            interval = factor(interval, levels = c("default", "wide")))]

# Summary statistics, in the order they should be read: a bounded tail
```

```

# frequency first, then the mean the Jensen argument speaks to, then the
# median for comparability, then the diagnostic that says whether the mean can
# be taken at face value.
summarise_e <- function(d) {
  d[, .(tail_frac = mean(F_hat > E_TAIL_MULT * om$F_true),
        tail_mcse = sqrt(mean(F_hat > E_TAIL_MULT * om$F_true) *
                          (1 - mean(F_hat > E_TAIL_MULT * om$F_true)) / .N),
        mean_bias = mean(F_hat) / om$F_true - 1,
        mean_mcse = stats::sd(F_hat) / sqrt(.N) / om$F_true,
        bias      = median(rel_bias),
        mcse      = mcse_median(F_hat) / om$F_true,
        bound_pct = 100 * mean(at_bound),
        n        = .N), by = .(NO_e)][order(NO_e)]
}

summ_e      <- summarise_e(res_e[interval == "default"])
summ_e_wide <- summarise_e(res_e[interval == "wide"])

knitr::kable(
  summ_e[, .(NO_e, tail_frac, tail_mcse, mean_bias, mean_mcse,
             bias, mcse, bound_pct, n)],
  digits = 4,
  col.names = c("Tags per group", paste0("P(F-hat > ", E_TAIL_MULT, "x truth)"),
                "MCSE (tail)", "Mean rel. bias", "MCSE (mean)",
                "Median rel. bias", "MCSE (median)", "% at bound", "n"),
  caption = paste0("flag0 against release-group size, at psi = ",
                    round(psi_of(om, PHI_E, NMIX_E), 3),
                    " (phi = ", PHI_E, ", nmix = ", NMIX_E,
                    ", one release group), over the default search interval.")
)

```

Table 19: flag0 against release-group size, at psi = 0.519 (phi = 2, nmix = 4, one release group), over the default search interval.

Tags per group	P(F-hat > 5x truth)	MCSE (tail)	Mean rel. bias	MCSE (mean)	Median rel. bias	MCSE (me- dian)	% at bound	n
125	0.06	0.0119	0.9174	0.1996	-0.0483	0.2502	4.25	400
250	0.01	0.0050	0.1695	0.0419	0.0051	0.0525	0.25	400
500	0.00	0.0000	0.1014	0.0252	0.0045	0.0316	0.00	400
2000	0.00	0.0000	0.0236	0.0110	0.0085	0.0137	0.00	400
10000	0.00	0.0000	0.0005	0.0048	-0.0011	0.0060	0.00	400

The final two columns are the ones to read first. % at bound is the proportion of replicates whose estimate is sitting on an endpoint of the search interval; where it is non-zero, the mean in that row is partly a statement about where the endpoint was put. The comparison against the wider interval below is what settles whether it matters.

```
stab_e <- merge(
  summ_e[, .(NO_e, tail_d = tail_frac, mean_d = mean_bias,
             med_d = bias, bound_d = bound_pct, mcse_d = mean_mcse)],
  summ_e_wide[, .(NO_e, tail_w = tail_frac, mean_w = mean_bias,
                 med_w = bias, bound_w = bound_pct, mcse_w = mean_mcse)],
  by = "NO_e")

knitr::kable(
  stab_e[, .(NO_e, tail_d, tail_w, mean_d, mean_w, med_d, med_w,
            bound_d, bound_w)],
  digits = 4,
  col.names = c("Tags per group", "Tail (default)", "Tail (wide)",
                "Mean (default)", "Mean (wide)", "Median (default)",
                "Median (wide)", "% bound (default)", "% bound (wide)"),
  caption = paste0("The same design fitted over the default search interval ",
                  "and over a much wider one. Statistics that move between ",
                  "the two columns are properties of the ceiling rather than ",
                  "of the estimator."))
```

Table 20: The same design fitted over the default search interval and over a much wider one. Statistics that move between the two columns are properties of the ceiling rather than of the estimator.

Tags per group	Tail (de- fault)	Tail (wide)	Mean (de- fault)	Mean (wide)	Median (default)	Median (wide)	% bound (default)	% bound (wide)
125	0.06	0.0625	0.9174	0.9900	-0.0483	-0.0483	4.25	4.00
250	0.01	0.0100	0.1695	0.1694	0.0051	0.0051	0.25	0.25
500	0.00	0.0000	0.1014	0.1014	0.0045	0.0045	0.00	0.00
2000	0.00	0.0000	0.0236	0.0236	0.0085	0.0085	0.00	0.00
10000	0.00	0.0000	0.0005	0.0005	-0.0011	-0.0011	0.00	0.00

Boundary contact is not negligible: at the smallest group size 4.2% of replicates return an estimate sitting on an endpoint of the default interval. The mean at that cell is therefore a censored quantity, and the wide-interval column is the one to read. Widening the interval moves the mean at the smallest group size by +7.3%, against a joint Monte Carlo standard error of 29.5%, so the mean is stable to the choice of ceiling. The gross-overshoot frequency

moves by +0.2% — as it should, since the threshold of $5\times$ the truth lies far below either ceiling — which is why it is the statistic carried into the summary sections.

```
# Three statistics on a shared x axis, free y scales, so the tail frequency is
# not squashed by the scale of the mean. One ggplot rather than a stacked
# layout, to avoid a package dependency the rest of the document does not have.
e_panel <- rbind(
  summ_e[, .(NO_e, stat = paste0("P(F-hat > ", E_TAIL_MULT, "x truth)"),
    val = tail_frac, mcse = tail_mcse)],
  summ_e[, .(NO_e, stat = "Mean relative bias",
    val = mean_bias, mcse = mean_mcse)],
  summ_e[, .(NO_e, stat = "Median relative bias",
    val = bias, mcse = mcse)])

e_panel[, stat := factor(stat, levels = c(
  paste0("P(F-hat > ", E_TAIL_MULT, "x truth)"),
  "Mean relative bias", "Median relative bias"))]

ggplot(e_panel, aes(NO_e, val)) +
  geom_hline(yintercept = 0, linetype = 2, colour = "grey40") +
  geom_linerange(aes(ymin = val - 2 * mcse, ymax = val + 2 * mcse),
    alpha = 0.7, colour = pal[["flag0"]]) +
  geom_line(linewidth = 0.7, colour = pal[["flag0"]]) +
  geom_point(size = 2.2, colour = pal[["flag0"]]) +
  facet_wrap(~ stat, ncol = 1, scales = "free_y") +
  # Break at the design points rather than at decade-ish defaults, which label
  # values the design never visits and leave the smallest group on the panel
  # edge.
  scale_x_log10(breaks = sort(unique(e_panel$NO_e)),
    labels = scales::comma,
    expand = expansion(mult = 0.07)) +
  scale_y_continuous(labels = scales::percent) +
  labs(x = "Tags per release group (log scale)", y = NULL)
```

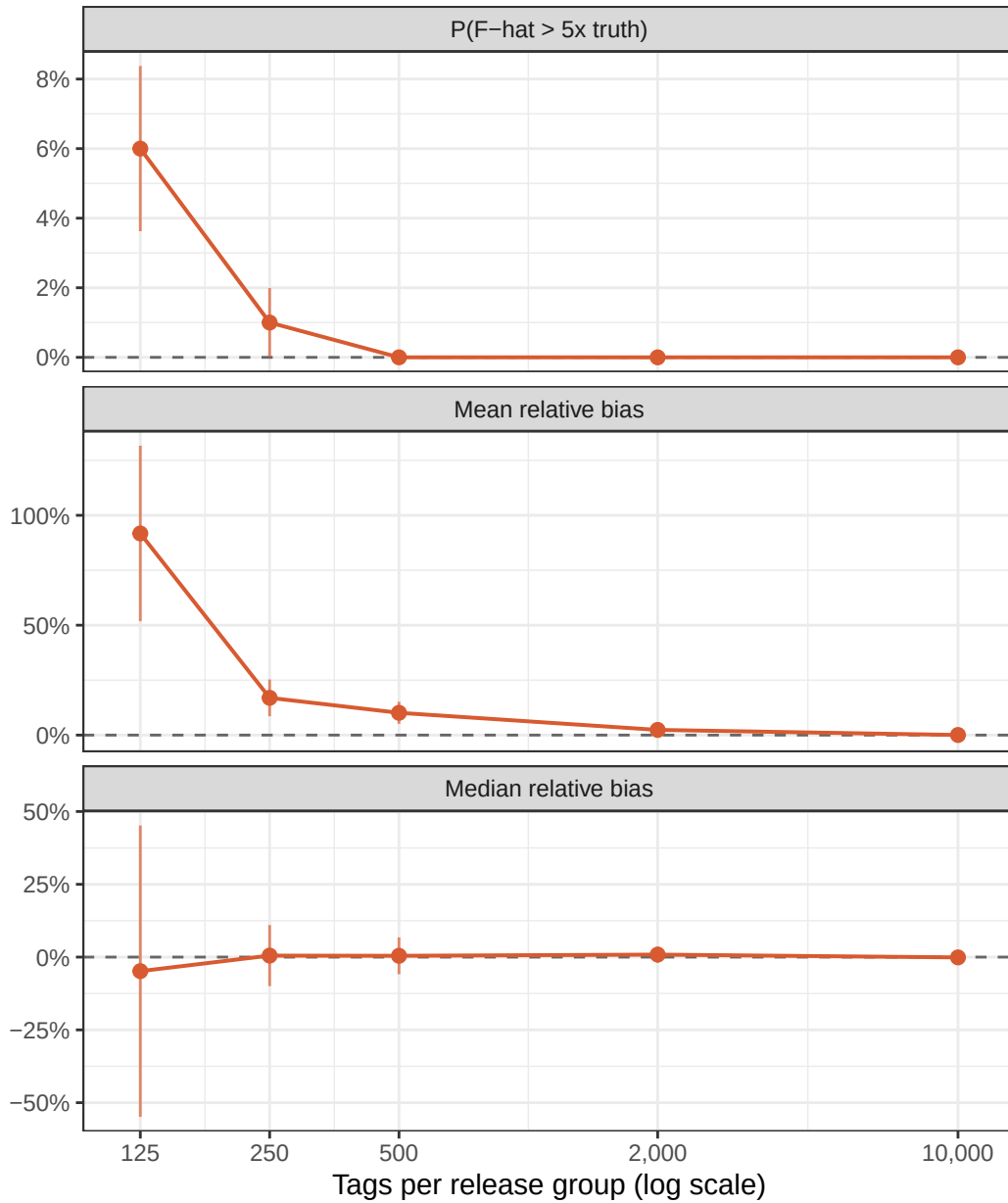


Figure 15: `flag0` against release-group size, with two Monte Carlo standard errors and free y scales. The reporting rate is correct and the survival floor is not engaging, so any departure from zero is attributable to release-group size alone. The top panel is the frequency of gross overshoot, which is a bounded proportion and insensitive to the search interval; the mean and median are shown separately below it because the effect appears in the tail frequency and the mean while the median is uninformative about it, which is the signature of a tail effect rather than a shift.

At 125 tags per group, 6.0% of replicates — 24 of 400 — return an estimate of F more than 5 times the truth; at 10,000 the figure is 0.0%. The difference is +6.0% against a joint Monte Carlo standard error of 1.2%, so the two ends of the design are separated by more than two joint standard errors. The corresponding difference in the mean is +91.7% against a joint standard error of 20.0%, which is also resolved and, per the interval check above, is not an artefact of the ceiling.

Across the design the correlation with $\log N_0$ is -0.67 on the overshoot frequency and -0.71 on the mean. The median behaves differently — its correlation is 0.52 — but the median at the smallest group size is not distinguishable from zero (-4.8%, against a Monte Carlo standard error of 25.0%), so it carries no information about the effect in either direction. **The honest reading is that the median is blind to this effect, not that it contradicts it.** That is what the mechanism predicts. A convex inversion applied to a noisier target moves probability mass into the right tail; it does not shift the bulk of the distribution the same way, so the centre stays where it was while the mean is dragged upward by the minority of groups that report many returns during mixing. Any diagnostic that looks only at the middle of the distribution will see nothing at all, when what is actually happening is a heavy right tail.

The effect is resolved, and it runs in the direction the convexity argument predicts. That makes release-group size a third condition on the default’s unbiasedness, independent of the reporting rate and the survival floor: the deterministic inversion is unbiased in the large-sample limit, not in general. The form the failure takes is a tail rather than a shift — at 125 tags per group roughly one release group in 17 returns an estimate more than 5 times the truth, while the median at that cell is not distinguishable from zero — so assessments built from many small release groups should not expect the Experiment B robustness result to hold at face value, and should not expect a median-based diagnostic to warn them.

The three statistics have to be read together, and none of them is sufficient alone. At 125 tags per group the median is not distinguishable from zero, the mean is +91.7%, and the overshoot frequency is 6.0%. A single small release group can therefore look unremarkable on a median-based diagnostic while contributing a large upward pull to a pooled estimate. The mean overstates the typical group, the median conceals the tail entirely, and the frequency is the statistic that says what a reader of a real assessment should actually expect to encounter: not that every small group is biased, but that some fraction of them are badly so. The obvious lever is the assumed reporting rate: at fixed N_0 , a lower $X_{assumed}$ thins R_t directly and inflates the divisor, attacking both sides of the ratio at once. That is listed under [Suggested extensions](#) rather than run here, because the low- X region is also where the survival floor engages, and separating the two mechanisms needs a design that holds the floor inactive by construction.

Note that this experiment isolates group size from the survival floor — the reporting rate is correct, so the raised target is correct on average and the floor is not the operative mechanism. In a real configuration the two run together, and small groups make floor engagement more likely as well.

8 What the results show

1. When the assumed reporting rate is right, `flag0` is unbiased and `flag1` is not — and this holds across the whole Experiment B configuration space. `flag0` recovers F regardless of φ and `nmix` (largest absolute median bias 2.7%), because the deterministic inversion never needs to know how available tagged fish were. `flag1` asserts $X = 1$ during mixing, which is false for every tagging programme ever run; its bias in F is downward, meaning biomass biased *upward*, reaching -63% in the worst cell, and it worsens steeply as either the mixing period lengthens or tagged fish become more vulnerable. `tag_flags(i,2) = 1` is a robustness choice, not a correctness fix, and it errs optimistically.

Experiments C and E qualify this heavily. The unbiasedness of `flag0` survives a reporting rate that is correct only on average rather than group by group, and it survives clumped reporting *provided the survival floor does not engage*. Where it does — low assumed reporting rates, high availability — `flag0`'s median bias reaches -36%, because a capped removal is an under-removal and under-removal is `flag1`'s error. Experiment E adds a third condition: the result is a large-sample one, and it degrades at realistic release-group sizes. The claim that the default is unbiased holds only inside a region of configuration space, and that region shrinks as assumed reporting rates fall.

2. The reporting rate, not the flag, is the first-order channel — except at high availability. The `forward` estimator handles mixing correctly but keeps X fixed at the wrong value, so the gap between it and `flag0` measures what the mixing treatment contributes over and above the reporting-rate error. Across the low-availability part of the design that gap is small: at $\varphi = 0.5$ it never exceeds 2.7%, and outside the `true > assumed` column it never exceeds 0.4%. A wrong reporting rate rescales every predicted return in the *informative* periods too, not just the mixing ones, and that channel dominates.

The exception matters. The largest gap in the design is 63%, at $\varphi = 2$, `nmix` = 4, `true > assumed` — +139% for `flag0` against +76% for `forward`. Where tagged fish are highly available during a long mixing period, the deterministic removal stops being a second-order correction and becomes a large error in its own right. So the flag is not *always* the minor channel; it is minor in the regime the mixing period was designed for ($\varphi < 1$) and major in the regime that is arguably more realistic for purse-seine tagging. Experiment C shows the same boundary in a different coordinate: the gap is negligible where the floor is inactive and first-order where it is not. Attributing a biomass shift entirely to the flag will not survive a sensitivity run, but nor will dismissing it without knowing where in φ the fishery sits.

3. The two flag settings form a one-sided interval and cannot bracket the truth when the assumed reporting rate is too high. Because $X_{\text{assumed}} < 1$, `flag0` always removes more tags than `flag1` and always returns the larger $\hat{F}$; the interval therefore has `flag0` as its upper end. It can only contain the truth when `flag0` sits above it, which happens only when the true reporting rate is at or above the assumed value. Across the entire 27-cell design, bracketing never occurs in any replicate whenever the true rate is below the assumed

one. Where bracketing does occur it is either degenerate (`true = assumed`, where `flag0` is simply unbiased and noise straddles the truth roughly half the time) or genuine but extremely wide (high φ , `true > assumed`, intervals up to 1.8 times F).

Implication for grid design: running `tag_flags(i,2)` at 0 and 1 as alternative levels spans an assumption; it does not bound the truth, and the direction in which it fails to bound depends on the sign of the reporting-rate error — which is exactly the thing under dispute. If the real uncertainty is in the reporting rate, that is what belongs on the grid.

4. Longer mixing periods amplify a reporting-rate error under every estimator except `forward_X`. This is a correction to an expectation worth stating explicitly, because the intuition that “the forward model turns excluded data into information, so longer mixing should help” is not what the simulation shows. At $\varphi = 2$ with the rate underestimated, `flag0` runs $+68\% \rightarrow +139\%$ as `nmix` goes $2 \rightarrow 4$. But `forward` degrades too ($+55\% \rightarrow +76\%$ over the same range), because fixing X at the wrong value contaminates the extra mixing-period observations just as it contaminates the post-mixing ones. Adding data does not help when the added data are being scored against a misspecified nuisance parameter.

Only `forward_X`, which lets X move, is flat in `nmix` ($+1\% \rightarrow +1\%$). The lesson is that handling the mixing period as a likelihood is necessary but not sufficient; the benefit only materialises when the reporting rate is estimable. Any sensitivity to `tag_flags(i,1)` in a real assessment should be read as partly a reporting-rate sensitivity in disguise.

5. Only estimating the reporting rate fixes the misspecification. `forward_X` recovers F to within 4.9% in 26 of the 27 cells of Experiment B — 23 of 27 at a 4% threshold — and recovers φ at all three values tested. The two largest residuals are +5.4% (`nmix` = 4, `phi` = 1, `true` = assumed) and -4.9% (`nmix` = 2, `phi` = 2, `true` < assumed), each within 2.2 Monte Carlo standard errors of zero, against a typical MCSE for this estimator of 1.9% — `forward_X` is much the noisiest of the four, so its worst cells are where noise would be expected to show. The structural reason it works at all is that under `flag0` and `flag1` the reporting rate deterministically rewrites the tagged population and the data have no mechanism to object; under the forward parameterisation the mixing-period returns become evidence about X rather than an instruction. Note the caveat in the Limitations: this vignette identifies X far more cleanly than MULTIFAN-CL could.

6. The default’s failure mode is real, and it is triggered by low assumed reporting rates in combination with high availability, not by uncertainty in the rate. The floor engages when the realised catch fraction in a mixing period exceeds $(1 - \tau)X_{\text{assumed}}$, about 0.78 times the assumed rate under MULTIFAN-CL defaults — a fifth easier to reach than the exhaustion condition $C_t/N_t > X_{\text{assumed}}$ that the obvious reading of the manual suggests. At an assumed rate of 0.5 with $\varphi = 1$ that is not reached in this design; at 0.5 with $\varphi = 2$ it is reached in 3–67% of replicates, and at 0.2 with $\varphi = 2$ in 57–100%.

Trip-level clumping *amplifies* this but is not required for it. Small release groups and stochastic recaptures are sufficient on their own: with trips taking only 1% of a release group — effectively independent reporting — the floor still engages in 17% of replicates at $X_{\text{assumed}} = 0.2$, $\varphi = 1$,

and 57% at $\varphi = 2$. This matters for how the result is defended: clumping is the ingredient an unsympathetic reader will call contrived, and the finding does not depend on it. An assessment that revises its tag-seeding estimates downward moves itself into this regime without changing anything else.

7. Under floor engagement, the default degenerates into the alternative. Where the floor rarely engages, clumped reporting costs `flag0` precision but not accuracy — the median stays within 4% while the interquartile range roughly doubles. Where it routinely engages, `flag0` loses accuracy too, and in the same direction as `flag1`: once `posfun` caps the removal the default cannot remove the number of tags its own rule demands, so it under-removes and F is biased downward (median -36% at $X_{assumed} = 0.2$, $\varphi = 2$). The practical consequence is that the two settings are not as different as they look in the regime where the choice matters most. Neither behaviour would be visible in standard assessment diagnostics, which is an argument for checking the floor condition directly rather than inferring it from fit statistics.

8. `flag1`'s bias is a function of one measurable quantity. The mixing-period recapture fraction ψ collapses φ and `nmix` onto a single axis, and the analytic approximation $-(1 - X)\psi N_0 / (N_{post} + (1 - X)\psi N_0)$ tracks the simulation closely (correlation 0.998). This makes the question empirical rather than general: compute ψ from the `.tag` file and the expected downward bias in F follows — a few percent at $\psi = 0.05$, roughly a third of F at $\psi = 0.3$, more than half at $\psi = 0.5$.

9. The likelihood-based treatment is not free. `forward_X` recovers F and X cleanly in Experiments A and B, where one release group of 10,000 tags carries the information. In Experiment C's more realistic configuration — eight groups of 500 — it is the widest of the four estimators in 89% of cells and its median wanders over -46% to +35%. Structural correctness does not substitute for information: estimating the reporting rate, the availability multiplier and F together needs data that a fragmented tagging programme may not provide. This tempers the recommendation in point 5, and it is a reason to treat a code change as a medium-term improvement rather than a fix available to the current assessment.

10. Release-group size is a structural assumption. Experiment E shows the default's unbiasedness is a large-sample property: inverting a noisy target is convex in the target, so small release groups put weight in the upper tail of F even when the reporting rate is correct and the floor is inactive. At 125 tags per group, 6.0% of groups return an estimate more than 5 times the truth, against 0.0% at 10,000, while the median at the small-group cell is not distinguishable from zero. The failure is a tail, not a shift, which is why it is invisible to the diagnostics an assessment normally reports. Release-group size is not usually treated as an assumption worth stating; on this evidence it is one.

9 What to check in an assessment that fits to tags

This is the practical residue of the five experiments, in the order it is worth working through. It sits here rather than in the executive summary because every figure it refers to has been produced by the time it renders, so the numbers below are read from the fitted objects rather than transcribed.

1. Which setting was used, and whether it was held constant. `tag_flags(i,2)` was inert in MULTIFAN-CL until version 2.2.7.6 (July 2025), so any comparison against an earlier assessment involves a model change rather than a like-for-like retrospective. “They used the default” describes the behaviour correctly but not a decision anyone made — the alternative did not exist. A change of setting between a draft and a final model would move biomass in a predictable direction and should be documented as a structural choice, not a bug fix.

2. The assumed reporting rates, and the direction of any revision. This is the first-order channel through most of the design: at $\varphi = 0.5$ the gap between the deterministic treatment and a correctly specified forward model never exceeds 2.7%. A downward revision of tag-seeding estimates raises inferred F and lowers biomass through the ordinary likelihood, *and* separately pushes the model toward the regime where the default breaks down. The two effects compound and are easily conflated. The exception is high availability during a long mixing period, where the flag becomes a first-order channel in its own right (largest gap in the design 63%, at $\varphi = 2$, `nmix` = 4). For the 2026 bigeye assessment the values and the direction of revision are both documented: SC22-SA-IP-05 gives purse seine reporting-rate priors with means of 0.496 in region 2, 0.512 in region 3 and 0.528 in region 4, each between 15 and 24 per cent below what the 2023 assessments would have used under the same region and fishery definitions, with the reduction driven largely by the introduction of double-tag effects. Read those as penalty distributions rather than fixed values — the model estimates the rate against them — and note that they cover purse seine fleets only, that the region 2 penalty was deliberately halved to reflect additional uncertainty, and that the region 2 prior is not recommended for the Indonesian and Philippine domestic fisheries.

3. Whether the survival floor is engaging. The floor engages when the realised catch fraction in a mixing period exceeds $(1 - \tau)X_{assumed}$, about 0.78 times the assumed rate under MULTIFAN-CL defaults — that is 0.390 at an assumed rate of 0.5 and 0.156 at 0.2, which ordinary fishing mortalities reach. When it happens, `posfun` silently caps the removal and the default then **under-removes**, biasing F downward and biomass upward — the same direction as the alternative it is supposed to improve on. In MULTIFAN-CL the floor is set by `age_flags(118)` and (119), with a penalty weighted by `age_flags(117)/1e8`; if `age_flags(117) < 0` the penalty weight is zero and the rescaling leaves no trace in the objective function at all. Check `age_flags(117)`, (118) and (119), and check whether `age_flags(198)` (release-group-specific reporting rates) is active, which makes the divisor vary by group.

4. The mixing-period recapture fraction ψ . For `tag_flags(i,2) = 1` the downward bias in F is largely governed by this one measurable quantity: the proportion of each release

group *recaptured* within `tag_flags(i,1)` periods of release. The `.tag` file records reported returns rather than recaptures, so the returns have to be raised before dividing:

$$\hat{\psi}_i \approx \frac{1}{X} \cdot \frac{\sum_{t \leq \text{tag_flags}(i,1)} R_{it}}{N_i}.$$

Omitting the $1/X$ returns $X\psi$ rather than ψ , which at a reporting rate near 0.5 halves the diagnosed fraction — and because the bias steepens with ψ , it understates the consequence by more than half again. Then read the bias off Experiment D or evaluate $-(1-X)\psi N_0/(N_{post} + (1-X)\psi N_0)$ directly: -4.8% to -3.9% at $\psi = 0.05$; -33.4% to -27.7% at $\psi = 0.3$; -59.9% to -50.9% at $\psi = 0.50$. Each range is printed worst-first, and because a longer mixing period carries the worse bias at a given ψ — the ordering the Experiment D tripwire guards — the worst end is the longest mixing length. Take that end where `tag_flags(i,1)` is large. Two caveats: $\hat{\psi}$ inherits any error in the reporting rate it is raised by, in the direction that understates the problem; and a release-group-weighted mean aggregates over groups whose mixing lengths differ. This is an afternoon’s work against the `.tag` file and it turns a general argument into a specific one about that assessment’s data.

5. The size of the release groups. The default inverts a noisy observation, and that inversion is convex in the observation, so small release groups put weight in the upper tail of F that the large-sample argument does not predict. At 125 tags per group, 6.0% of release groups return an estimate more than 5 times the truth with the reporting rate correct and the floor inactive, against 0.0% at 10,000. Note that the median at the small-group cell is not distinguishable from zero: the cost is a heavy right tail across release groups, not a shift every group shares, so it will not show up in a diagnostic built on medians. Count the releases per group in the `.tag` file and check how much of the tagging data sits in groups of a few hundred.

6. Whether the flag appears as a grid axis, and how it is described. Running both settings spans an assumption; it does not bound the truth. The pair is structurally one-sided — the default always returns the larger F — so it can never bracket when the assumed reporting rate is too high, and where it does bracket the interval reaches 1.8 times the quantity being estimated. If the real uncertainty is in the reporting rate, that is what belongs on the grid.

10 Guidance on `tag_flags(i,2)`

Neither setting of `tag_flags(i,2)` is correct, and the choice between them should be made and reported as a conditional one.

Simulation testing shows that the default, `tag_flags(i,2) = 0`, recovers fishing mortality without appreciable bias when the assumed tag reporting rate is correct, across all combinations of mixing-period length (2–4 quarters) and tagged-fish availability ($\varphi = 0.5$ –2) tested in

Experiment B. That robustness is structural: the deterministic inversion solves for whatever mixing-period fishing mortality reproduces the implied removal, so it does not require the availability of tagged fish during mixing to be correctly specified.

It is, however, conditional on further requirements, and the claim should not be extended past the design in which it was demonstrated. Where the realised catch fraction in a mixing period exceeds roughly four-fifths of the assumed reporting rate, the raised removal drives the tagged population below the survival floor, the floor engages, and the default then under-removes — biasing fishing mortality downward by as much as 36% in our tests. This condition is reached routinely at assumed reporting rates around 0.2, and at 0.5 whenever tagged fish are substantially more available during mixing than the general population. The default’s reliability therefore declines precisely as reporting-rate estimates are revised downward — and they are being revised downward. The purse seine reporting-rate priors for bigeye in SC22-SA-IP-05 have means of 0.496 to 0.528 across regions 2 to 4, between 15 and 24 per cent below the values the 2023 assessments would have used under the same region and fishery definitions. The 2026 bigeye assessment therefore sits at the boundary of this condition, not safely inside it, and a further revision of the magnitude just observed would move it across. Separately, the result is a large-sample one: at 125 tags per release group, 6.0% of release groups return an estimate of fishing mortality more than 5 times the truth even with a correct reporting rate and an inactive floor, while the median at the same cell is not distinguishable from zero. The cost is a heavy right tail across release groups rather than a shift each group shares, so it is invisible to a median-based check and it does not average away as quickly as pooling suggests.

The alternative, `tag_flags(i,2) = 1`, assumes complete reporting during the mixing period. This is not true of any tagging programme, and the resulting bias is systematic rather than conservative: fishing mortality is underestimated and biomass correspondingly overestimated, with the error growing as either the mixing period lengthens or tagged fish become more available. Under plausible configurations the bias in fishing mortality reached 63%. Its one merit is predictability — the bias is largely a function of the proportion of each release group recaptured during the mixing period, which is computable from the `.tag` file in advance provided the reported returns are raised by the assumed reporting rate first. The qualifier matters in both directions: the recapture fraction does not determine the bias entirely, since at a fixed fraction a longer mixing period is worse by 9.1% at the top of the range tested; and an analyst who forgets the raising will diagnose half the fraction and conclude the setting is far cheaper than it is.

On balance the default remains preferable where reporting rates are moderate, tagged-fish availability during mixing is not elevated, the survival floor is demonstrably not engaging, and release groups are large enough for the large-sample argument to apply. All four conditions should be checked and reported rather than assumed. Neither setting protects against misspecification of the reporting rate itself, which is the larger error channel in most cases examined; the two settings do not bracket the truth and should not be used as alternative levels in a structural uncertainty grid. Where reporting rates are uncertain, that uncertainty should be represented directly.

The treatment that would resolve the dilemma rather than manage it is to predict mixing-period returns and score them in the likelihood, with the reporting rate estimated against its tag-seeding prior. MULTIFAN-CL cannot currently do this — mixing-period returns do not enter the tag likelihood under either setting — so it is a request to the MULTIFAN-CL development team rather than something an assessment can configure. We recommend it as a medium-term change, with the caveat that the simulation here also shows it is not free: with many small release groups it was the least precise of the four estimators tested, under a prior looser than any assessment would impose.

Because both settings are biased and the choice is unavoidable in any model fitted to tags, we further recommend that a configuration omitting tagging data — with spatial structure carried by a fleets-as-areas approach — be included in the structural uncertainty grid. This bounds the influence of the tag data on stock status without requiring the mixing-period treatment to be resolved first, and it answers a question of direct management relevance: how much of the assessment result depends on the tagging programme. This recommendation follows from the demonstrated deficiencies of both flag settings rather than from any simulation of a spatial model, which is outside the scope of this vignette.

11 Limitations

This is a deliberately stripped-down operating model and it flatters the forward model in ways a real assessment would not.

- **The number of tags released is known exactly**, and there is no abundance, selectivity, movement, or catchability to confound with the reporting rate. Identification of X here is far stronger than in MULTIFAN-CL, where X trades off against q and against absolute abundance. The extent to which `forward_X` rescues a bad prior is therefore optimistic, and the prior standard deviation (0.4 on the log scale) is loose. Tightening it toward a hard tag-seeding constraint moves `forward_X` back toward `forward`.
- **φ is a single scalar applied uniformly across the mixing period.** In reality availability presumably decays smoothly from release, and varies by fishery. A step function is the simplest possible caricature.
- **No spatial structure.** The reason a mixing period exists at all is that effort is not randomly distributed relative to release sites; that spatial process is represented here only through its effect on φ .
- **Observation error.** Experiments A, B, D and E use unclumped reporting; Experiment C adds trip-level clumping but the likelihood remains Poisson, whereas MULTIFAN-CL offers negative binomial and binned options via `parest_flags(111)`. Fitting a likelihood matched to the clumping would narrow the dispersion penalty seen in Experiment C.
- **The posfun analogue is approximate, and its penalty does no work.** The smooth floor and quadratic penalty reproduce the qualitative behaviour of `do_newton_raphson_for_tags()` — catch capped, survival floored — but it is not a

transcription of the MULTIFAN-CL code, and the weight is set by hand rather than derived from `age_flags(117)`. More importantly, because the mixing target depends only on the data and not on any parameter, the penalty is constant with respect to the parameter vector and cannot influence $\hat{F}$. That makes this vignette a faithful model of the `age_flags(117) < 0` case specifically. A configuration with a positive penalty weight might behave differently, and nothing here tests that.

- **Clumping is modelled as a single trip size.** Real reporting varies by fleet, and a mixture of well-reporting and non-reporting fleets would behave differently from a homogeneous trip process with the same mean. At the coarsest trip setting the realised number of trips per release group per period falls to a handful, so that corner of the design should be read as an extreme rather than as a typical fishery.
- **The forward estimators are not tuned for the Experiment C sample size.** `forward_X` estimates three parameters from eight groups of 500 tags under a loose prior; its poor performance there reflects that configuration rather than a defect of the parameterisation. A tighter prior, more release groups, or fewer free parameters would all improve it, and this vignette does not locate the boundary.
- **Experiment C fixes the mixing length at `nmix = 3`.** Experiment B's claim of robustness across mixing lengths is therefore not re-tested under the realistic observation model.
- **Experiment E resolves a difference, not a curve, and the mean it reports is not a bias correction.** The design separates the smallest and largest release-group sizes by 6.0% in overshoot frequency, against a joint Monte Carlo standard error of 1.2%, which establishes that group size matters but does not pin down the functional form between the two ends. The mean at the smallest group size is driven by a small number of extreme replicates and carries a large standard error of its own (20.0%); it is stable to a large widening of the search interval, but it remains a statistic about a tail and should be read as evidence of one rather than as a number to subtract off an assessment. The solved F is unbounded above in principle, so any mean of it is conditional on a bound imposed by the analyst – which is a general property of the deterministic inversion, not an artefact of this simulation. Release-group size also varies total tags alongside group size, so precision varies across the design, which is why every statistic is reported with its own Monte Carlo standard error.
- **No spatial or fleets-as-areas configuration is simulated.** The recommendation to include a tag-free branch in a structural uncertainty grid follows from the demonstrated deficiencies of both flag settings, not from any result in this document.
- **ψ is treated as known.** In practice it would be estimated from observed recaptures, which are themselves thinned by the reporting rate, so a real calculation would need to raise them — reintroducing some of the uncertainty the exercise is trying to characterise.

12 Suggested extensions

- Replace the Poisson observation model with a negative binomial and estimate the dispersion, to see how much of the contrast survives realistic overdispersion.
- Let φ decay geometrically with quarters at liberty rather than switching off at `nmix`, and estimate the decay rate. This is closer to the biology and would show whether `tag_flags(i,1)` needs to be a sharp boundary at all.
- Add multiple release cohorts with φ grouped by quarter at liberty — the structure that makes φ identifiable in a real model, and worth demonstrating before proposing it to the developers.
- **Sweep the prior standard deviation on X in `forward_X`, and treat this as the highest-value outstanding run in this list.** The value used throughout is 0.4 on the log scale. The penalty parameters in SC22-SA-IP-05 imply coefficients of variation on the bigeye purse seine reporting rate of roughly 0.05 to 0.09 — close to an order of magnitude tighter than the prior imposed here. `forward_X` was the least precise estimator tested in this document, and that finding is one of the reasons given for treating a likelihood-based treatment as a medium-term development request rather than a current option. Until the sweep is run it is not possible to say how much of the imprecision is intrinsic to the estimator and how much is an artefact of a prior far looser than any assessment would impose. The sweep would also locate where the ability to revise the reporting rate is lost, which is the other half of the trade-off.
- Fit with `nmix` misspecified relative to the truth, since in practice the analyst chooses `tag_flags(i,1)` without knowing the real mixing time.
- Add a `forward` variant with φ fixed at 1. The current `flag0-forward` gap conflates two changes — scoring the mixing-period returns in the likelihood, and adding a free availability parameter — so it does not cleanly isolate the mixing channel.
- Give the survival-floor penalty something to act on, by making the mixing-period target depend on an estimated quantity, and check whether a positive `age_flags(117)` changes the Experiment C conclusions.
- **Establish what bounds the solved F in MULTIFAN-CL.** In this vignette the deterministic inversion is unbounded above and is held in check by an explicit search interval, which is why every tail statistic here is reported against a boundary-contact diagnostic. A real implementation has its own effective bound — through parameter bounds on F , through the survival floor, or through the Newton solver’s own failure behaviour — and which of these binds first determines what an extreme release group actually does to an assessment. Reading that out of the code would say whether the tail found in Experiment E is realised as a large $\hat{F}$, as a floored removal, or as a convergence failure the analyst would notice.
- Vary `nmix` within Experiment C, to test whether Experiment B’s robustness across mixing lengths survives the realistic observation model.
- **Push the noise-to-signal ratio through $X_{assumed}$ as well as through release-group size,** to separate the two routes into the same mechanism. A lower assumed

rate thins R_t and inflates the divisor simultaneously, so it attacks the ratio harder than N_0 can alone, and it is the axis a real assessment moves along when seeding estimates are revised. The complication is that the low- X region is also where the survival floor engages, so the design would need to hold the floor inactive by construction — a large N_0 with a low φ , or the floor disabled outright — to attribute any effect to the inversion rather than to the cap. Done properly this would also pin down the functional form that Experiment E establishes the existence of but does not resolve.

- Raise the replicate count in Experiment E and add intermediate group sizes. The present design establishes that the two ends differ but leaves the shape between them unresolved, and a tail frequency is an inherently noisy statistic: its Monte Carlo error scales as $n^{-1/2}$, so halving the resolvable difference from the present 2.4% costs four times the replicates. That is affordable here, since the flag modes are one-dimensional, and it is what would turn the third condition from a qualitative warning into something an analyst could calibrate against their own release-group sizes.

13 Session information

```
sessionInfo()
```

```
R version 4.5.2 (2025-10-31 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26100)
```

```
Matrix products: default
LAPACK version 3.12.1
```

```
locale:
[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8
```

```
time zone: Pacific/Honolulu
tzcode source: internal
```

```
attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods   base
```

```
other attached packages:
[1] ggplot2_4.0.3      magrittr_2.0.4      data.table_1.16.2
```

loaded via a namespace (and not attached):

[1] vctrs_0.7.3	cli_3.6.5	knitr_1.51	rlang_1.2.0
[5] xfun_0.57	otel_0.2.0	generics_0.1.4	S7_0.2.1
[9] jsonlite_2.0.0	labeling_0.4.3	glue_1.8.0	htmltools_0.5.8.1
[13] scales_1.4.0	rmarkdown_2.31	grid_4.5.2	tibble_3.3.1
[17] evaluate_1.0.5	fastmap_1.2.0	yaml_2.3.12	lifecycle_1.0.5
[21] compiler_4.5.2	dplyr_1.2.1	RColorBrewer_1.1-3	pkgconfig_2.0.3
[25] farver_2.1.2	digest_0.6.37	R6_2.6.1	tidyselect_1.2.1
[29] pillar_1.11.1	withr_3.0.2	tools_4.5.2	gtable_0.3.6